

Imperial College London

MENG INDIVIDUAL PROJECT

IMPERIAL COLLEGE LONDON

DEPARTMENT OF COMPUTING

Systematic Benchmarking of Monte Carlo Engines with Automatic Differentiation Across Modern Architectures

Author:
Vivian Lopez

Supervisor:
Prof. William Knottenbelt

Second Marker:
Prof. Ben Glocker

June 10, 2026

Abstract

Monte Carlo (MC) simulation with automatic differentiation (AD) is widely used in quantitative finance and scientific computing, where both numerical estimates and parameter sensitivities must be computed efficiently. However, the performance characteristics of MC+AD workloads are difficult to evaluate systematically. Runtime, scalability, memory behaviour, and numerical stability depend not only on the underlying stochastic model, but also on differentiation mode, execution backend, compiler/runtime behaviour, threading strategy, and hardware configuration. Existing studies often evaluate these factors in isolation, making it difficult to determine which software and hardware choices are appropriate for a given workload and deployment budget.

This project develops a reproducible benchmarking and profiling framework for MC+AD option-pricing workloads across modern software and hardware environments. The framework provides explicit workload specifications, a common engine interface, controlled warm-up and repeated measurement, deterministic random seeding, numerical validation against reference solutions, structured result storage, and environment metadata capture. Using this framework, a set of representative workloads is evaluated across Python/NumPy, JAX, C++, and Rust implementations on both local and cloud-based hardware platforms.

The experimental study focuses on workload regimes rather than isolated headline timings. Constant-volatility European options are used as a baseline terminal-payoff workload, while parametric local-volatility models introduce additional arithmetic complexity and state-dependent dynamics. A path-dependent Asian option workload is further used to stress memory usage and expose differences in backend scaling behaviour. Across these workloads, the project investigates how backend rankings change with workload scale, how forward- and reverse-mode AD alter runtime and memory costs, and how cloud hardware selection affects cost-performance trade-offs.

The primary contribution of the project is a workload-aware profiling methodology for MC+AD systems. Rather than treating backend and hardware selection as a naive exhaustive grid search problem, the proposed approach uses short probe runs to identify dominated configurations and approximate the runtime-cost Pareto frontier before performing more expensive benchmark evaluations. This reframes cloud resource selection as a reproducible, budget-aware systems problem and enables more efficient exploration of hardware/software design choices.

The results demonstrate that no single backend or hardware configuration is uniformly optimal across all workloads. Instead, effective deployment depends on workload structure, differentiation requirements, execution scale, and cost constraints. Overall, the project contributes both a practical benchmarking framework and an empirical methodology for analysing MC+AD workloads, providing evidence-based guidance for the design and selection of efficient Monte Carlo simulation systems with automatic differentiation.

Acknowledgements

I would like to thank my supervisor, William Knottenbelt, for his helpful guidance and advice during the project. His feedback helped clarify the direction of the work and identify the areas where the project could be strengthened.

I am also grateful to the HSBC Quant team for their support and industrial perspective. In particular, I would like to thank Zouhair Rajehi for acting as the main point of contact and for providing useful background, technical context, and pointers for the project. I would also like to thank Renaud X. Delloye for his input during the meetings, and Ouejdane Ghodhbane for coordinating the meetings and providing a welcoming point of contact with the team.

Finally, I would like to thank the Department of Computing at Imperial College London for the opportunity to undertake this project.

Contents

1	Introduction	7
1.1	Motivation	7
1.2	Problem Statement	7
1.3	Research Questions	8
1.4	Contributions	8
1.5	Report Structure	8
2	Background and Related Work	9
2.1	Monte Carlo Simulation	9
2.2	Automatic Differentiation	9
2.3	Monte Carlo + AD Challenges	10
2.4	Existing Frameworks	10
2.5	Hardware Considerations	10
2.6	Budget-Aware Benchmark Selection	11
2.7	Summary of Gaps	11
3	System Design	13
3.1	Design Goals	13
3.2	High-Level Architecture	14
3.3	Layer 1 — Workload Configuration	14
3.3.1	Workload 1 — European Option	15
3.3.2	Workload 2 — European Local-Volatility Option	15
3.4	Layer 2 — Engine Interface	16
3.5	Layer 3 — Benchmark Runner	17
3.6	Layer 4 — Storage	18
3.7	Layer 5 — REST API	18
3.8	Layer 6 — Frontend	19
3.9	Cloud Cost-Performance Design	19
3.10	Budget-Aware Profiler Design	19
3.10.1	Probe-Based Baseline	20
3.10.2	Successive Halving Profiler	20
3.11	Design Decisions and Trade-offs	20
3.12	Summary	21
4	Implementation	22
4.1	Overview	22
4.2	Core Modules	22
4.2.1	Workload Registry	22
4.2.2	Monte Carlo Engine Interface	23
4.2.3	NumPy Engine	23
4.2.4	JAX Engine	23
4.2.5	C++ Engine	24
4.2.6	Rust Engine	24
4.2.7	CUDA Engine	25
4.2.8	Benchmark Runner	25
4.2.9	Metrics Collection	26
4.2.10	Storage Layer	26

4.2.11	REST API	26
4.2.12	Frontend / Dashboard	26
4.3	Supported Workloads	27
4.4	AD Integration	27
4.5	GPU Support	27
4.6	Challenges	27
4.7	Cloud Metadata and Cost Support	28
4.8	Profiler Implementation	28
4.9	Testing	29
4.10	Dependencies	29
4.11	Summary	29
5	Experimental Setup	30
5.1	Overview	30
5.2	Connection to the Research Questions	30
5.3	Execution Environment	30
5.4	Workloads	32
5.4.1	European Option under Geometric Brownian Motion	32
5.4.2	European Option under Parametric Local Volatility	32
5.4.3	Asian Arithmetic-Average Option	33
5.5	Engines and Software Stacks	33
5.6	Experimental Variables	33
5.6.1	Independent Variables	33
5.6.2	Dependent Variables	34
5.6.3	Controlled Variables	34
5.7	Metrics	34
5.8	Measurement Methodology	35
5.8.1	Warmup and JIT Compilation	35
5.8.2	Repeated Measurements	35
5.8.3	Randomness and Correctness	35
5.8.4	Fairness Controls	36
5.8.5	Data Exclusion Policy	36
5.9	Experiment Scripts	36
5.10	Individual Experiment Designs	36
5.10.1	European GBM Baseline	36
5.10.2	European GBM AD Overhead and Greek Accuracy	37
5.10.3	European GBM Path-Count Scalability	37
5.10.4	European GBM Thread Scalability	38
5.10.5	Local-Volatility Benchmark Suite	38
5.11	Final Cloud and Profiler Experiments	39
5.11.1	Full-Grid Cloud Baseline	39
5.11.2	Old Profiler Baseline	39
5.11.3	Successive Halving Profiler	40
5.12	Execution Order	40
5.13	Database Queries for Analysis	40
5.13.1	European Baseline Throughput	40
5.13.2	European AD Overhead and Greek Accuracy	41
5.13.3	Local-Volatility Engine Scaling	41
5.13.4	Local-Volatility AD Overhead	41
5.13.5	Thread Scaling	42
5.14	Reproducibility	42
5.15	Scope of Excluded Extensions	42
5.16	Summary	43

6	Results and Discussion	44
6.1	Overview	44
6.2	European GBM Validation	44
6.3	Local-Volatility Smoke Test	44
6.4	Local-Volatility Engine Scalability	46
6.5	Local-Volatility AD Scaling with Path Count	46
6.6	Local-Volatility Scaling with Time Steps	47
6.7	Local-Volatility AD Scaling with Time Steps	47
6.8	Memory Pressure and Practical Resource Limits	47
6.9	Thread Scalability on Local Volatility	48
6.9.1	Strong Scaling	48
6.9.2	Weak Scaling	48
6.10	Surface-Shape Sensitivity	49
6.11	Timing Stability	49
6.12	Stress Limit	49
6.13	Final Cloud Cost-Performance and Profiler Results	50
6.13.1	Correctness and AD Overhead in the Final Cloud Run	50
6.13.2	Cloud Cost-Performance	51
6.13.3	Pareto Frontier	51
6.13.4	Old Profiler versus SHA	51
6.13.5	SHA Progression	52
6.14	Cross-Question Discussion	52
6.14.1	Software Stack and Engine Performance	52
6.14.2	Programming-Language and AD Trade-offs	53
6.14.3	Parallelism and Bottlenecks	53
6.14.4	AI/ML Compiler Techniques	53
6.14.5	Scope Against the Original Proposal	54
6.15	Limitations and Threats to Validity	54
6.15.1	Hardware and Cloud Scope	54
6.15.2	Local-Volatility Correctness	54
6.15.3	Memory Measurement	54
6.15.4	Thread Control	54
6.15.5	Excluded Proposal Extensions	54
6.16	Summary	55
7	Conclusion	56
7.1	Summary of Work	56
7.2	Key Findings	56
7.3	Answering Research Questions	56
7.4	Future Work	57
7.5	Final Remarks	57
	Declarations	58
8	Appendix	59
8.1	Additional Results	59
8.2	Configuration Files	59
8.3	Extended Tables	59
8.4	Code Snippets	59

List of Figures

3.1	High-level system architecture and data flow.	14
6.1	Runtime and estimated cost per run for best no-AD cloud configurations at $M = 100,000$	51
6.2	Runtime versus estimated cost for the final n2-standard-4 grid. Pareto-optimal points are highlighted.	52
6.3	SHA active configuration count as probe path budget increases.	53

List of Tables

3.1	Fields of <code>EuropeanOptionConfig</code>	15
3.2	Additional fields of <code>EuropeanLocalVolConfig</code>	15
3.3	REST API endpoints.	19
5.1	Mapping from research questions to local experiment evidence.	31
5.2	Engine configurations used in the local experimental programme.	33
5.3	Core experiment scripts and their roles.	37
6.1	Result sections and their corresponding evidence.	45
6.2	European GBM baseline validation at $M = 10,000$	45
6.3	LV-0: smoke test ($M = 1,000$, $N = 252$, one timed run).	45
6.4	LV-1: engine scalability on local volatility ($N = 252$, no AD).	46
6.5	LV-2: JAX AD path-count scaling on local volatility ($N = 252$).	46
6.6	LV-3: time-step scaling on local volatility ($M = 50,000$, no AD).	47
6.7	LV-4: JAX AD time-step scaling on local volatility ($M = 25,000$).	47
6.8	LV-5: JAX memory-pressure experiment.	48
6.9	LV-6: strong thread scaling ($M = 100,000$, $N = 252$).	48
6.10	LV-7: weak thread scaling (base $M = 50,000$ paths/thread, $N = 252$).	48
6.11	LV-8: surface-shape sensitivity ($M = 50,000$, $N = 252$).	49
6.12	LV-9: timing stability ($M = 100,000$, $N = 252$, 20 timed runs).	49
6.13	LV-10: stress limit ($N = 252$, large M).	50
6.14	European GBM AD validation against Black–Scholes analytical values at $M = 100,000$	50
6.15	JAX automatic differentiation overhead at $M = 100,000$ on n2-standard-4.	50
6.16	Best no-AD cost-performance per priced cloud instance at $M = 100,000$	51
6.17	Full-grid, old-profiler and SHA comparison for the final n2-standard-4 cloud experiment.	52

Chapter 1

Introduction

1.1 Motivation

Monte Carlo (MC) simulation is a foundational technique in scientific computing and quantitative finance, used to estimate expectations of stochastic systems where closed-form solutions are unavailable. In financial applications, MC methods are widely used for pricing complex derivatives and evaluating risk under uncertainty. However, modern use cases increasingly require not only accurate estimates, but also sensitivities of these estimates with respect to model parameters, such as Greeks in option pricing.

Automatic differentiation (AD) provides a principled and efficient way to compute such sensitivities by differentiating the numerical program itself. Unlike finite difference methods, AD produces derivatives that are accurate up to floating-point precision and integrates naturally with simulation code. The combination of MC and AD (MC+AD) therefore enables flexible, gradient-based analysis of stochastic systems.

Despite its advantages, MC+AD introduces significant computational challenges. These workloads are typically compute-intensive, memory-sensitive, and highly dependent on implementation details such as hardware architecture, programming language, numerical libraries, and differentiation mode. As high-performance computing (HPC) systems become increasingly heterogeneous—spanning CPUs, GPUs, and cloud-based environments—understanding how MC+AD workloads behave across these systems becomes critical.

This project is motivated by the need to systematically evaluate and optimise MC+AD workloads in modern computing environments, with a focus on performance, scalability, and cost-efficiency.

1.2 Problem Statement

While Monte Carlo simulation, automatic differentiation, and high-performance computing have each been studied extensively, there is a lack of unified frameworks and methodologies for evaluating their combined behaviour. Existing work often focuses on isolated aspects, such as MC algorithms, AD systems, or hardware benchmarking, without considering their interactions in a controlled and reproducible manner.

In particular, several gaps can be identified. First, there is no standardised benchmarking framework that supports MC workloads with integrated AD across multiple programming languages and hardware architectures. Second, most AD benchmarking studies are conducted in the context of machine learning, leaving stochastic simulation workloads comparatively underexplored. Third, performance comparisons across systems are often difficult to interpret due to inconsistent experimental setups, lack of reproducibility, and insufficient reporting of configuration details.

Furthermore, modern optimisation techniques developed in the AI/ML community—such as operator fusion, mixed precision, and hardware-aware scheduling—have not been systematically evaluated in the context of MC+AD workloads. Finally, there is limited empirical guidance on selecting appropriate cloud hardware configurations for these workloads, particularly when considering cost-performance trade-offs.

As a result, practitioners lack clear, evidence-based insights into how to design, implement, and deploy efficient MC+AD systems.

1.3 Research Questions

This project addresses the following research questions:

- How do different hardware architectures and software stacks affect the throughput and efficiency of MC+AD workloads?
- What are the trade-offs between programming languages and AD libraries in terms of performance, maintainability, and parallelism?
- Which Google Cloud hardware configurations provide the best cost/performance for large-scale MC+AD tasks?
- How can we leverage the latest AI community techniques to improve training/inference and maximize hardware utilization?
- How do parallelization strategies impact scaling, and where do bottlenecks arise?

1.4 Contributions

This project makes the following contributions:

- The design and implementation of a modular, extensible benchmarking framework for Monte Carlo engines with automatic differentiation support.
- A systematic methodology for evaluating MC+AD workloads across different hardware architectures, software stacks, and programming languages.
- An experimental study comparing performance, scalability, and memory behaviour under varying configurations, including differentiation modes and parallelisation strategies.
- An analysis of cost-performance trade-offs for MC+AD workloads in cloud computing environments, including measured Google Cloud runs and per-run cost estimates.
- A budget-aware profiling methodology based on short probe runs and successive halving, evaluated against a full-grid cloud baseline.
- Insights into the applicability and limits of AI-inspired compiler/runtime techniques, particularly JAX/XLA, for stochastic simulation workloads.

The framework and methodology developed in this work provide a foundation for reproducible and extensible benchmarking of MC+AD systems. The final evaluation combines detailed local CPU experiments with cloud cost-performance and profiler-selection experiments.

1.5 Report Structure

The remainder of this report is organised as follows.

- Chapter 2 reviews the background and related work, including Monte Carlo simulation engines, automatic differentiation, benchmarking methodologies, and relevant techniques from high-performance computing and machine learning.
- Chapter 3 describes the system design of the benchmarking framework, including workload definitions, execution model, data collection pipeline, cloud metadata model, and budget-aware profiling design.
- Chapter 4 describes the concrete implementation of the framework, including the engine modules, benchmark runner, storage layer, API, dashboard, cloud-cost metadata, and profiler driver.
- Chapter 5 outlines the experimental methodology, covering validation, performance benchmarking, scalability analysis, cloud experiments, and profiler-versus-grid comparisons.
- Chapter 6 presents and analyses the experimental results, focusing on performance trends, AD overheads, scaling limits, cloud cost-performance, Pareto frontiers, and profiler quality.
- Chapter 7 concludes the report, summarising key findings and discussing potential directions for future work.

Chapter 2

Background and Related Work

2.1 Monte Carlo Simulation

Monte Carlo (MC) simulation is a widely used technique for estimating expectations of stochastic systems through repeated random sampling. It is particularly valuable in settings where analytical solutions are unavailable or intractable, such as the pricing of financial derivatives under complex dynamics. In such applications, MC methods approximate quantities of interest by simulating a large number of independent sample paths and averaging the resulting payoffs.

A key property of MC simulation is that its statistical error decreases at a rate proportional to $O(N^{-1/2})$, where N is the number of samples. As a result, achieving high accuracy typically requires a large number of simulations, making computational efficiency a primary concern. At the same time, MC workloads are often described as “embarrassingly parallel”, since individual simulation paths can be evaluated independently. This makes them well-suited to modern parallel hardware, including multi-core CPUs and GPUs.

In practice, Monte Carlo *engines* extend basic simulations by providing configurable, reusable implementations that support parameter variation, reproducibility through controlled random number generation, and repeated execution for statistical aggregation. Benchmarking such engines therefore requires not only measuring raw execution time, but also understanding how performance scales with workload size, parallelism, and implementation choices.

2.2 Automatic Differentiation

Automatic differentiation (AD) is a technique for computing derivatives of numerical programs by systematically applying the chain rule to the sequence of elementary operations that define the computation. Unlike finite difference methods, AD produces derivatives accurate up to floating-point precision while avoiding numerical instability.

Two primary modes of AD are commonly used. Forward-mode AD propagates derivatives alongside values during the forward execution of a program and is typically efficient when the number of input parameters is small relative to the number of outputs. In contrast, reverse-mode AD records intermediate computations during a forward pass and then propagates sensitivities backwards, making it more efficient when a small number of outputs depend on many inputs.

Modern AD systems, such as those integrated into scientific computing and machine learning frameworks, enable gradients to be computed directly from program implementations. This is particularly useful in simulation-based contexts, where sensitivities (e.g., financial “Greeks”) can be obtained without manual derivation.

However, the choice between forward- and reverse-mode AD introduces important trade-offs in computational cost and memory usage. Forward-mode typically incurs moderate overhead with minimal additional memory requirements, while reverse-mode can significantly increase memory usage due to the need to store intermediate states. These trade-offs become especially important in large-scale simulations.

2.3 Monte Carlo + AD Challenges

Combining Monte Carlo simulation with automatic differentiation enables flexible and general sensitivity analysis, but introduces several non-trivial computational challenges.

First, the computational cost of AD compounds with the cost of simulation. In MC workloads, each sample path may consist of many time steps and numerical operations. Applying AD to such computations increases the amount of work performed per path, with the overhead depending on the differentiation mode and implementation. As a result, throughput (e.g., paths per second) can be significantly reduced compared to non-differentiated simulations.

Second, memory usage becomes a critical concern, particularly for reverse-mode AD. Reverse-mode requires storing intermediate values during the forward execution in order to compute gradients during the backward pass. In long-horizon simulations with many time steps, this can lead to substantial memory growth, potentially exceeding cache capacity and degrading performance due to increased memory traffic. In extreme cases, memory limitations may prevent scaling to larger workloads altogether.

Third, the interaction between AD and parallelism is complex. While Monte Carlo simulation is naturally parallel across independent paths, AD introduces dependencies within each path that may limit opportunities for parallel execution. Furthermore, reverse-mode AD can introduce additional synchronisation points and data movement, particularly in GPU or distributed settings, reducing the effectiveness of parallelisation strategies.

Finally, the combination of stochastic computation and differentiation introduces challenges for reproducibility and numerical stability. Ensuring that differentiated simulations produce consistent and meaningful results requires careful control of random number generation, seeding, and execution order.

These challenges motivate the need for systematic evaluation of MC+AD workloads, where both computational performance and memory behaviour are measured under controlled and reproducible conditions.

2.4 Existing Frameworks

Several frameworks exist for Monte Carlo simulation and high-performance numerical computing, though few explicitly target the combination of MC and AD across heterogeneous environments.

Carlo.jl is a notable example of a Monte Carlo framework designed for high-performance simulations in Julia. It provides abstractions for defining simulation workloads, managing parallel execution, and handling reproducibility through structured configuration and checkpointing. Its design emphasises separation between the Monte Carlo algorithm and the surrounding execution infrastructure, enabling reuse and composability of simulation components.

However, Carlo.jl is intentionally scoped to a single language ecosystem and does not directly address cross-language benchmarking, automatic differentiation, or heterogeneous hardware support. While it demonstrates effective scheduling strategies and reproducible execution, it does not provide a general framework for comparing implementations across different programming languages, AD systems, or hardware architectures.

More broadly, existing benchmarking efforts in scientific computing and machine learning often focus on specific domains or workloads, such as linear algebra kernels or neural network training. While these benchmarks provide valuable insights into hardware and compiler performance, they do not capture the unique characteristics of MC+AD workloads, particularly the interaction between stochastic simulation, differentiation, and parallel execution.

This highlights the need for a benchmarking framework that treats Monte Carlo simulation and automatic differentiation as first-class concerns, while enabling fair and reproducible comparison across implementations.

2.5 Hardware Considerations

The performance of Monte Carlo and MC+AD workloads is strongly influenced by the underlying hardware architecture. While MC simulations are often considered compute-intensive, in practice their performance can be limited by a combination of arithmetic throughput, memory bandwidth, and parallel execution efficiency.

On modern CPUs, performance depends on effective utilisation of vectorisation (SIMD), multi-threading, and cache hierarchies. Poor memory locality or inefficient vectorisation can significantly reduce achievable performance, even for otherwise simple workloads. In the presence of reverse-mode AD, increased memory usage can exacerbate these issues by placing additional pressure on caches and memory bandwidth.

GPUs offer high throughput for data-parallel workloads and are well-suited to large-scale Monte Carlo simulations. However, their performance depends on careful management of memory access patterns, kernel launch overheads, and control-flow divergence. The introduction of AD can further complicate GPU execution, as differentiation may require additional kernel passes, increased memory usage, or limited compiler optimisation opportunities.

In heterogeneous and cloud environments, additional factors such as data transfer overheads, resource contention, and cost efficiency become relevant. High raw performance does not necessarily translate to optimal cost-performance, particularly when specialised hardware incurs higher operational costs.

Understanding these hardware-level effects is essential for interpreting benchmark results and identifying performance bottlenecks in MC+AD workloads.

2.6 Budget-Aware Benchmark Selection

Full-grid benchmarking is attractive because it gives a direct comparison of all candidate workloads, engines, AD modes, and hardware choices. However, it becomes expensive when each candidate must be evaluated at production-scale path counts on cloud instances. A practical benchmarking system therefore needs not only accurate measurement, but also a way to decide which configurations deserve the most expensive measurements.

This motivates the use of cheap probe runs. A probe run evaluates a candidate configuration at a smaller path count or reduced budget and uses that result as evidence about whether the configuration is likely to remain competitive at full scale. The central risk is selection error: a configuration that is slow at low budget may become competitive at high budget, or a noisy probe may give an incorrect ranking. Any profiler must therefore be evaluated against a full-grid baseline rather than assumed to be correct.

Successive halving is a natural fit for this setting. It begins by evaluating all candidates at a low budget, promotes a smaller active set to a larger budget, and repeats until only the strongest candidates remain for full evaluation. In the context of MC+AD benchmarking, the budget can be interpreted as the number of simulated paths. This connects the statistical structure of Monte Carlo experiments with the systems problem of cloud cost control: most configurations receive only cheap measurements, while promising configurations receive enough budget to support reliable final decisions.

2.7 Summary of Gaps

Despite extensive work in Monte Carlo simulation, automatic differentiation, and high-performance benchmarking, several gaps remain in the literature.

First, there is a lack of standardised benchmarking frameworks that support Monte Carlo workloads with integrated automatic differentiation across multiple programming languages and execution environments. Existing tools typically focus on a single language or domain, limiting their ability to support fair cross-system comparisons.

Second, the performance characteristics of AD in stochastic simulation contexts are not well understood. While AD has been extensively studied in machine learning, its interaction with Monte Carlo workloads—particularly in terms of memory usage, scaling behaviour, and parallel execution—has received comparatively little attention.

Third, existing benchmarks often fail to capture the full range of factors influencing performance, including hardware configuration, software stack, differentiation mode, and execution strategy. This makes it difficult to draw general conclusions or reproduce results across different environments.

Finally, there is limited support for integrating performance, resource utilisation, and cost considerations into a unified evaluation framework. For practical applications, particularly in

cloud environments, understanding cost-performance trade-offs is as important as measuring raw execution speed.

This project addresses these gaps by developing a modular, language-agnostic benchmarking framework for MC+AD workloads, enabling systematic and reproducible evaluation across hardware architectures, software stacks, and differentiation strategies. The framework is designed to expose non-obvious trade-offs in performance, scalability, and resource usage, providing insights that are not readily obtainable from existing tools or benchmarks.

Chapter 3

System Design

3.1 Design Goals

The framework is designed around four guiding principles: *reproducibility*, *extensibility*, *fairness*, and *traceability*. Together these principles constrain almost every interface, data structure and naming convention in the codebase.

Reproducibility. Every benchmark run is fully described by a single, serialisable `WorkloadConfig` object. The `config_hash()` method serialises this object into canonically sorted JSON and computes a truncated (8-character) SHA-256 digest. The hash is stable across machines and Python versions and is used to group repeated runs of the same experiment for statistical comparison.

All sources of randomness are explicitly seeded. For a given engine on a given machine, the same configuration produces bit-identical prices and paths. Different engines use different RNG implementations (NumPy `default_rng`, JAX `PRNGKey`, C++ `mt19937_64`, Rust `SmallRng`, CUDA `curand`), so the same seed produces statistically equivalent but numerically distinct results across engines — this is an intentional design choice rather than a defect, and is documented in the codebase. Cross-engine validation is therefore performed against the Black-Scholes analytical solution rather than against bit-equal outputs.

Extensibility. The framework is organised around two minimal abstractions: `WorkloadConfig` (what to simulate) and `MonteCarloEngine` (how to simulate it). Adding a workload requires only declaring a new `@workload`-decorated dataclass. Adding an engine requires only subclassing `MonteCarloEngine` and implementing `run()`, `supports()` and `supported_ad_modes()`. A central `WORKLOAD_REGISTRY` maps string identifiers to workload constructors and is consumed by both the REST API and the React frontend, so new workloads appear automatically in the UI without backend–frontend duplication.

Fairness. All engines are evaluated under a single benchmarking protocol implemented in `BenchmarkRunner`. Each trial consists of a configurable warmup phase whose results are discarded, followed by a fixed number of timed repetitions. Warmup is essential for JIT-compiled backends: a cold JAX call at $M = 10,000$ takes approximately 115 ms, compared to approximately 0.6 ms once XLA has cached the compiled binary. Excluding the cold compile cost from timing isolates the steady-state computational performance that is relevant in a production setting.

Traceability. Every row in the SQLite database carries the full serialised configuration (`config_json`), the analytical reference values, all timing statistics, and a complete snapshot of the execution environment (Python version, platform, CPU model and core count, RAM, NumPy version, JAX version). Any historical run can be reconstructed exactly, even if the corresponding `WorkloadConfig` class is later modified.

3.2 High-Level Architecture

The framework follows a strict four-layer architecture, with two additional layers providing the REST API and the React frontend. Each layer depends only on the layers below it, so any component can be replaced or extended in isolation.

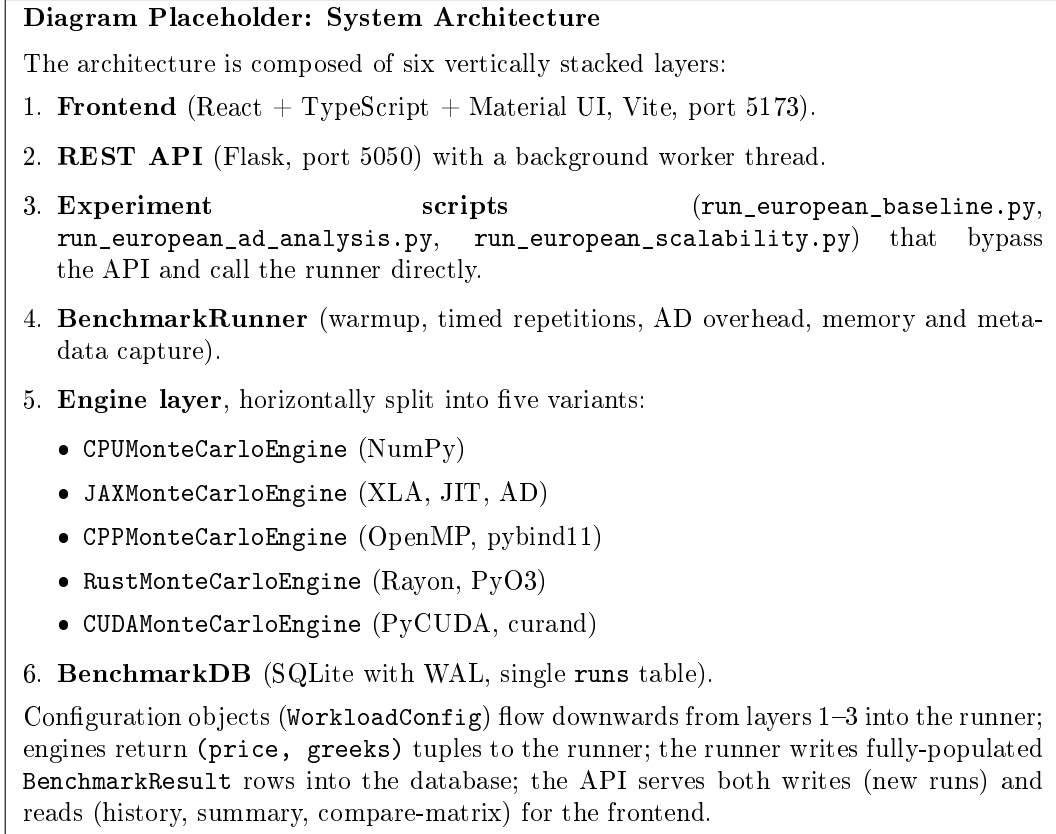


Figure 3.1: High-level system architecture and data flow.

3.3 Layer 1 — Workload Configuration

The workload layer uses a registry pattern implemented via a single Python decorator. The `@workload(name)` decorator simultaneously applies `@dataclass` to the class, sets a class-level `workload_type` attribute, and registers the class in a global `WORKLOAD_REGISTRY` dictionary. Adding a new workload therefore requires only writing a new class: no boilerplate, no changes to the runner, storage, API or frontend.

The base class `WorkloadConfig` provides four concrete inherited methods used by every workload:

- `validate()` — a no-op by default, overridden for cross-field constraints (e.g. enforcing $\sigma > 0$, $T > 0$, valid `option_type`).
- `to_dict()` — full serialisation via `dataclasses.asdict()` with `workload_type` injection.
- `from_dict()` — safe deserialisation, ignoring unknown keys so that schema migrations do not break older saved configurations.
- `config_hash()` — the 8-character SHA-256 fingerprint of the sorted serialised configuration.

There are no abstract methods on `WorkloadConfig`; every subclass inherits working implementations. The free function `config_from_dict()` is the single deserialisation entry point: it reads the `workload_type` key, looks up the corresponding class in `WORKLOAD_REGISTRY`, and delegates to that class's `from_dict()`.

3.3.1 Workload 1 — European Option

`EuropeanOptionConfig` (`workload_type = "european"`) describes a plain vanilla European call or put under constant-volatility geometric Brownian motion (GBM). Its fields are summarised in Table 3.1.

Table 3.1: Fields of `EuropeanOptionConfig`.

Field	Default	Description
<code>S0</code>	100.0	initial spot price
<code>K</code>	100.0	strike price
<code>r</code>	0.05	continuously-compounded risk-free rate
<code>sigma</code>	0.20	annualised volatility
<code>T</code>	1.0	time to maturity (years)
<code>option_type</code>	"call"	"call" or "put"
<code>N</code>	1	number of time steps (single-step exact)
<code>M</code>	10 000	number of Monte Carlo paths
<code>seed</code>	42	RNG seed for reproducibility

The pricing model uses single-step exact GBM. The terminal stock price is

$$S_T = S_0 \exp\left((r - \tfrac{1}{2}\sigma^2)T + \sigma\sqrt{T}Z\right), \quad Z \sim \mathcal{N}(0,1),$$

and the option price is the discounted expected payoff. This is the reference workload for all engine and AD comparisons because Black–Scholes provides an exact analytical benchmark for both prices and Greeks.

Analytical validation is built directly into the framework. The `CPUMonteCarloEngine` module exposes Black–Scholes closed-form functions for the call/put price, Delta, Vega and Rho:

$$\Delta_{\text{call}} = \Phi(d_1), \quad \text{Vega} = S_0 \phi(d_1) \sqrt{T}, \quad \rho_{\text{call}} = K T e^{-rT} \Phi(d_2),$$

where $d_1 = (\ln(S_0/K) + (r + \frac{1}{2}\sigma^2)T) / (\sigma\sqrt{T})$ and $d_2 = d_1 - \sigma\sqrt{T}$. These values are computed and stored alongside every Monte Carlo result, enabling automatic absolute and relative error metrics.

3.3.2 Workload 2 — European Local-Volatility Option

`EuropeanLocalVolConfig` (`workload_type = "european_local_vol"`) extends the European workload to a multi-step local-volatility model. The configuration adds the parameters in Table 3.2.

Table 3.2: Additional fields of `EuropeanLocalVolConfig`.

Field	Default	Description
<code>M</code>	100 000	paths (larger than European for multi-step precision)
<code>N</code>	252	Euler steps (daily over one year)
<code>sigma_min</code>	0.01	volatility floor
<code>theta</code>	auto	polynomial parameters $[a_0, a_1, a_2, b_1]$

The default θ is set in `__post_init__` so that $\sigma_{\min} + \text{softplus}(a_0) = 0.20$, i.e. the model reduces to a 20% flat-volatility GBM. The closed-form solution is $a_0 = \log(\text{expm1}(0.20 - \sigma_{\min}))$. When $a_1 = a_2 = b_1 = 0$, the model collapses to constant-volatility GBM and the Monte Carlo price converges to the Black–Scholes price as M and N increase. This provides an automatic regression test embedded in the default configuration.

The dynamics are

$$dS = r S dt + \sigma(S, t; \theta) S dW,$$

discretised in log-space using a log-Euler scheme:

$$S_{n+1} = S_n \exp\left((r - \tfrac{1}{2}\sigma_n^2) \Delta t + \sigma_n \sqrt{\Delta t} Z_n\right).$$

At each step the local volatility is computed from a parametric monomial surface:

$$x_n = \ln(S_n/S_0), \quad \text{raw}_n = a_0 + a_1 x_n + a_2 x_n^2 + b_1 t_n, \quad \sigma_n = \sigma_{\min} + \text{softplus}(\text{raw}_n).$$

The softplus activation is implemented in a numerically stable form $\max(\text{raw}, 0) + \log(1 + \exp(-|\text{raw}|))$, which avoids overflow for large positive inputs and catastrophic cancellation for large negative inputs. This formulation is reproduced *identically* in NumPy, JAX, C++ and Rust so that the engines are differentiated only by their execution model, not by numerical behaviour.

The AD targets for this workload are Delta ($\partial P/\partial S_0$) plus the full θ -gradient $\partial P/\partial a_0$, $\partial P/\partial a_1$, $\partial P/\partial a_2$, $\partial P/\partial b_1$, $\partial P/\partial \sigma_{\min}$ — six inputs in total, computed in a single reverse-mode backward pass.

3.4 Layer 2 — Engine Interface

The `MonteCarloEngine` abstract base class defines a minimal interface:

```
class MonteCarloEngine:
    def run(self, config, ad_mode) -> tuple[float, dict | None]
    def supports(self, workload_type: str) -> bool
    def supported_ad_modes(self) -> tuple[str, ...]
```

The `ad_mode` argument takes one of three string values: `"none"`, `"forward"` or `"reverse"`. When `ad_mode` is `"none"` or the engine does not implement AD for the given workload, the returned `greeks` dictionary is `None`. The runner inspects `supports()` and `supported_ad_modes()` before dispatching a configuration, so engines that cannot service a request are silently skipped rather than raising errors.

Engine 1 — NumPy / CPU. `CPUMonteCarloEngine` is the single-threaded NumPy reference implementation. It supports both workloads and `ad_mode = "none"` only. For the European workload it uses `np.random.default_rng(seed)` to generate all M standard normals as a single array and evaluates the terminal price and payoff vectorially — no Python-level loop over paths. For the local-volatility workload it implements the N -step log-Euler scheme in Python, with each step operating vectorially on an array of M paths. The correctness of all other engines is verified against this implementation.

Engine 2 — JAX / XLA. `JAXMonteCarloEngine` is the only engine that exposes `ad_mode ∈ {none, forward, reverse}`. The European kernel is a *module-level* `@jax.jit` function decorated with `functools.partial(jax.jit, static_argnames=("option_type",))`. Defining the kernel at module scope ensures JAX traces and compiles it exactly once per `(shape, dtype, option_type)` signature; the random sample Z is passed as an argument rather than captured in a closure, so the compiled XLA computation is reused across every timed repetition. The local-volatility kernel uses `jax.lax.scan` to implement the N -step Euler loop, with `static_argnames=("option_type", "N")` so XLA emits specialised code for each call/put and step-count combination.

For the European workload, reverse-mode AD is implemented as a *single* `jax.grad(price_fn, args=(0,1,2))` call returning $(\partial P/\partial S_0, \partial P/\partial r, \partial P/\partial \sigma)$ in one backward pass. An earlier version invoked `jax.grad` three times with `args=0,1,2`, which was correct but tripled both compute time and peak memory — a result that materially distorts the AD overhead measurement. Forward-mode uses three `jax.jvp` calls probing the standard basis tangent vectors $(1, 0, 0)$, $(0, 1, 0)$ and $(0, 0, 1)$, exploiting the fact that forward mode is cheaper than reverse when the number of inputs is small relative to the number of outputs.

For the local-volatility workload, reverse-mode uses a single `jax.grad(price_fn, args=(0,1,2,3,4,5))` call that differentiates through the full N -step `scan`; forward-mode uses six `jax.jvp` calls. JAX differentiates through `jax.lax.scan` efficiently via its reverse-mode accumulation, avoiding the explosion in graph size that would occur if the loop were unrolled.

Engine 3 — C++ / OpenMP. `CPPMonteCarloEngine` is a thin Python wrapper around a native implementation in `cpu_engine.cpp` / `cpu_engine.hpp`, exposed via `pybind11` bindings. Each OpenMP thread owns its own `std::mt19937_64` generator seeded as `seed + thread_id`; an explicit `#pragma omp parallel` block with a reduction clause accumulates partial sums without locks. Results are fully deterministic for a given `(M, seed, thread_count)` triple; changing the thread count changes the per-path RNG assignment and produces statistically equivalent but numerically different results. The local-vol kernel reproduces exactly the same log-Euler discretisation and stable softplus formula as the NumPy and JAX engines.

Engine 4 — Rust / Rayon. `RustMonteCarloEngine` wraps a native implementation in `benchmarking/rust/src/lib.rs`, exposed via `PyO3` bindings and built with `maturin`. The engine uses `SmallRng` (Xoshiro128++) from the `rand` crate, with each Rayon worker thread seeded as `seed XOR thread_index` to mirror the C++ per-thread offset pattern. Sampling uses `rand_distr::StandardNormal`. The per-path simulation is a pure scalar loop with no heap allocation; the entire M -path simulation is expressed as a Rayon `into_par_iter` chain with `map_init` for per-thread RNG initialisation and a `sum` reduction. The softplus is implemented inline as `z.max(0.0) + (-z.abs()).exp().ln_1p()`.

Engine 5 — CUDA / PyCUDA. `CUDAMonteCarloEngine` compiles and launches CUDA kernels at runtime via `PyCUDA`. The engine is probed at server startup with a 128-path trial run; if CUDA initialisation fails it is silently marked unavailable and excluded from the engine registry. The kernel uses one thread per Monte Carlo path. Each thread initialises its own `curandState` via `curand_init(seed, thread_index, 0, &state)`, using the thread index as the sequence number so that statistical independence is guaranteed regardless of the block/grid layout. Each thread draws Z via `curand_normal_double`, computes the GBM terminal price, and writes the discounted payoff to a global output array which the host reduces with `np.mean`. Forward-mode Delta is implemented in a separate kernel using the pathwise estimator: $\partial S_T / \partial S_0 = S_T / S_0$, so the pathwise Delta is $\mathbf{1}_{\{S_T > K\}}(S_T / S_0)$ for a call and $-\mathbf{1}_{\{S_T < K\}}(S_T / S_0)$ for a put. The CUDA engine supports the European workload only.

3.5 Layer 3 — Benchmark Runner

`BenchmarkRunner` encapsulates the full benchmarking protocol. Given an engine instance and an optional display name, its `run()` method executes the following sequence:

1. Call `config.validate()`; abort early on failure.
2. Execute `num_warmup` warmup runs whose results are discarded; this excludes XLA compilation from timed measurements.
3. Execute `num_runs` timed runs, recording individual wall-clock times via `time.perf_counter()`.
4. If `ad_mode != "none"`, repeat steps 2–3 for a matched no-AD baseline (same engine, same config, identical warmup count) and compute the AD overhead ratio $r_{AD} = \bar{t}_{AD} / \bar{t}_{baseline}$.
5. Snapshot environment metadata: timestamp, Python version and implementation, platform, CPU model and architecture, logical CPU count, total RAM in GB, NumPy version, JAX version.
6. Aggregate timings into mean, standard deviation, min and max (in milliseconds), and compute `throughput_paths_per_sec = M / mean_runtime_s`.
7. Return a `BenchmarkResult` dataclass containing all of the above plus the price, the Greeks (if any), the analytical reference values, and the absolute and relative errors.

Memory peak is measured via `psutil`: the process resident-set size is snapshotted before and after the timed run set, and the peak is reported as `max(rss_during) - rss_before`. If `psutil` is unavailable the field is left blank, degrading gracefully.

3.6 Layer 4 — Storage

BenchmarkDB is a thread-safe SQLite wrapper using a per-call connection with `check_same_thread=False` and an explicit `threading.Lock` that serialises writes. The database is opened in Write-Ahead Logging (WAL) mode, which allows concurrent reads during writes — important because the Flask API serves read requests from the main thread while the background worker writes results. The database file lives at `results/benchmarks.db` and is created on first use.

Schema. The schema consists of a single `runs` table whose columns are defined declaratively by the `_FULL_SCHEMA_COLUMNS` list. Columns are grouped as follows:

- **Identity:** `id` (UUID, primary key), `experiment_id`, `experiment_type`, `config_hash`.
- **Status lifecycle:** `status` (pending/running/completed/failed), `error_message`, `created_at`, `started_at`, `completed_at` (all UTC ISO-8601).
- **Workload definition:** `workload_type`, `M`, `N`, `seed`, `config_json`.
- **Engine / implementation:** `engine`, `language`, `backend`.
- **Parallelism metadata:** `num_threads`, `vectorization_flag`, `batch_size`.
- **Runtime performance:** `mean_runtime_ms`, `std_runtime_ms`, `min_runtime_ms`, `max_runtime_ms`, `throughput_paths_per_sec`.
- **AD metadata:** `ad_mode`, `baseline_mean_ms`, `ad_overhead_ratio`.
- **Numerical results:** `result_value`, `greek_delta`, `greek_vega`, `greek_rho`.
- **Analytical reference:** `analytical_price`, `analytical_delta`, `analytical_vega`, `analytical_rho`.
- **Error metrics:** `abs_price_error`, `rel_price_error` and matching columns for Delta, Vega and Rho.
- **Resources:** `memory_peak_mb`.
- **Environment:** `cpu_model`, `cpu_architecture`, `cpu_count`, `memory_gb`, `platform`, `python_version`, `numpy_version`, `jax_version`, `blas_backend`.
- **Cloud and profiling:** `cloud_provider`, `instance_type`, `cloud_region`, `cloud_zone`, `hourly_price_usd`, `cost_per_run`, profiler-selection fields, and SHA promotion metadata where available.

Auto-migration. On startup, `_init_schema()` compares the existing table columns against the authoritative list using `PRAGMA table_info` and issues an `ALTER TABLE ADD COLUMN` for any missing column. Existing databases therefore continue to work unmodified as the schema evolves — a property that has proved valuable while iterating on the AD and cloud columns during development.

Write paths. Experiment scripts use `store_run_full()`, which inserts a fully-populated completed row in a single transaction, bypassing the pending/running state machine. The API uses the `create_run()` → `mark_running()` → `mark_completed()` lifecycle to support asynchronous execution.

3.7 Layer 5 — REST API

The Flask server (`benchmarking/api/server.py`, port 5050) is intentionally small. Flask-CORS is enabled for cross-origin requests from the frontend. A single daemon background thread polls BenchmarkDB for pending runs and executes them sequentially via `BenchmarkRunner`, transitioning the row through the `running` and `completed` (or `failed`) states. The endpoints are summarised in Table 3.3.

Engine availability is determined at startup. The C++ and CUDA engines may not be present in every environment; they are excluded from `/api/engines` without erroring if their initialisation probe fails.

Table 3.3: REST API endpoints.

Endpoint	Method	Purpose
/api/runs	POST	submit a new benchmark run
/api/runs	GET	list runs (filter by workload/engine/status)
/api/runs/:id	GET	poll a single run by UUID
/api/engines	GET	list available engines, workloads and AD modes
/api/workloads	GET	list registered workload types
/api/summary	GET	aggregate counts, fastest run, breakdowns
/api/compare_matrix	GET	matrix aggregated by (engine, ad_mode, config)

3.8 Layer 6 — Frontend

The frontend is a React + TypeScript single-page application built with Vite and styled with Material UI, listening on port 5173. It communicates with the Flask backend through a typed HTTP client in `src/api/client.ts`. The application has five pages:

- **SimulatePage** — submit a new run and poll for completion. The `SimulationForm` component is generated dynamically from the workload `SCHEMA` returned by the API, so new workloads appear automatically without frontend changes.
- **ComparePage** — engine comparison matrix backed by `/api/compare_matrix`.
- **HistoryPage** — filterable table of every run.
- **SummaryPage** — aggregate dashboard.
- **AnalysisPage** — AD overhead and Greek-accuracy analysis.

3.9 Cloud Cost-Performance Design

The original proposal emphasised cloud resource selection as a practical systems question rather than a purely local benchmark. The final design therefore extends each benchmark row with cloud metadata whenever the run is executed on a cloud instance. The relevant fields record the provider, instance type, region, zone, hourly price assumption, git commit, and derived cost per run. This keeps cost-performance analysis tied to the exact environment in which a run was measured.

The cost model is intentionally simple:

$$\text{cost per run} = \frac{\text{runtime seconds}}{3600} \times \text{hourly instance price.}$$

This does not attempt to model billing granularity, storage, data transfer, or queueing costs. Its purpose is to compare benchmark configurations under a common pricing assumption. This is sufficient for identifying whether a faster instance is also cheaper per run for the measured workload, while avoiding unsupported claims about long-term cloud bills or all future machine prices.

Cloud fields are stored alongside the same workload, engine, timing, numerical, and environment metadata as local runs. As a result, cloud and local experiments share the same analysis path: report artefacts can be regenerated from the database without manually copying console output into tables.

3.10 Budget-Aware Profiler Design

Full-grid benchmarking gives the cleanest reference point, but it scales poorly with the number of workloads, engines, AD modes, path counts, and instance types. The framework therefore includes profiler logic that uses cheaper probe runs to select a subset of configurations for full-scale benchmarking. The important design principle is that the profiler is not trusted by construction: it is evaluated against a full-grid baseline using explicit selection-quality metrics.

3.10.1 Probe-Based Baseline

The first profiler uses low-budget probes to rank all candidate configurations and then selects a conservative subset for full benchmarking. It includes diversity safeguards so that selection is not determined only by a single global top- k ranking. This makes it a useful baseline: it can save full runs, but it still evaluates a relatively large selected subset to protect against probe noise and workload-specific crossover behaviour.

3.10.2 Successive Halving Profiler

The successive-halving (SHA) profiler evaluates all candidate configurations at the cheapest probe budget, promotes a smaller active set to a larger probe budget, and repeats this process before choosing the final full-scale runs. In the final cloud experiment, the budget is the number of Monte Carlo paths, so the profiler asks whether small-path measurements are sufficiently predictive of full-path performance.

The design deliberately records intermediate SHA state rather than only the final selected set. For each promotion stage, the database or generated artefacts capture the active configuration count and probe budget. These records allow the report to show how many candidates are discarded at each stage and to measure the probe path budget consumed by the profiler.

Selection quality is measured using:

- **full runs saved**: the reduction in expensive full-scale benchmark evaluations relative to the full grid;
- **Pareto recovery**: the fraction of full-grid non-dominated configurations retained by the profiler;
- **runtime regret**: the percentage loss relative to the fastest full-grid configuration;
- **cost regret**: the percentage loss relative to the cheapest full-grid configuration; and
- **rank correlation**: Spearman agreement between probe rankings and full-scale rankings.

These metrics make the profiler result falsifiable. A profiler that saves runs but misses the Pareto frontier or introduces high regret would not be considered successful. Conversely, a profiler that preserves the full-grid decisions while reducing full runs contributes directly to the cloud resource-selection goal of the project.

3.11 Design Decisions and Trade-offs

Decorator-based workload registration. An earlier design made `WorkloadConfig` an abstract base class with abstract properties (`workload_type`, `M`, `seed`) and abstract methods (`validate()`, `to_dict()`, `from_dict()`). That design forced every workload to re-implement the same boilerplate and produced a class of bugs in which abstract method implementations drifted out of sync. The refactored design replaces all of this with a single `@workload(name)` decorator that wires up dataclass conversion, sets `workload_type`, and registers the class. New workloads need only declare their fields and optionally override `validate()`.

Module-level JAX kernels. Placing JAX kernels at module level (rather than as instance methods or nested functions) is critical for compilation caching. JAX traces and compiles based on function identity and on argument shapes and dtypes. If the kernel were a nested function inside `run()`, each call to `run()` would create a fresh function object and defeat the JIT cache, regenerating the XLA binary on every timed repetition.

Single-pass reverse-mode AD. Using a single `jax.grad(price_fn, argnums=(0,1,2))` call rather than three separate `jax.grad` calls is functionally equivalent but materially changes the measured AD overhead. The single-pass form shares the backward graph across all output partial derivatives and is consistent with how AD is deployed in production quant systems; this is the form benchmarked in this project.

jax.lax.scan for the local-volatility loop. The N -step Euler loop must be expressed as `jax.lax.scan` rather than as a Python `for` loop. A Python loop would cause JAX to unroll the entire computation during tracing, producing an XLA graph with $N = 252$ copies of the step body — both compile time and memory consumption grow with N . `jax.lax.scan` compiles to a single loop in the XLA graph and lets the compiler treat it as a unit.

SQLite over PostgreSQL. SQLite with WAL is sufficient for the expected single-machine workload (up to a few hundred runs per day). It requires no external service, ships as part of Python, and is straightforward to back up by copying a single file. WAL allows concurrent reads during writes, which is the only concurrency requirement imposed by the architecture. Should the framework be scaled out to a distributed Ray-based execution model (Stage 8 of the future-work plan), PostgreSQL would be a natural replacement.

Per-thread RNG streams. The C++ and Rust engines seed each worker thread independently. This sacrifices bitwise reproducibility across thread counts in exchange for lock-free parallel sampling. The framework records `num_threads` in the database so that runs at different thread counts are treated as statistically distinct configurations rather than as repeats.

3.12 Summary

The framework consists of six clearly separated layers: workload configuration, engine, runner, storage, API and frontend. The four implementation layers are strictly decoupled through two minimal abstractions (`WorkloadConfig` and `MonteCarloEngine`) and a registry-driven workload model. Every benchmark run is fully described by a serialisable configuration, fingerprinted by a deterministic hash, and persisted alongside its analytical reference values and execution environment, ensuring complete reproducibility and traceability. The next chapter describes how each layer is implemented in practice.

Chapter 4

Implementation

4.1 Overview

This chapter describes the concrete implementation of the framework laid out in Chapter 3. The codebase is hosted on GitHub at <https://github.com/Vivian-Lopez/Monte-Carlo-and-Automatic-Differentiation-Systematic-Benchmarking-Framework> and is organised as a single Python package with optional native subpackages. The top-level directory structure is:

```
benchmarking/  
  workloads/      # WorkloadConfig dataclasses + registry  
  engines/        # Python wrappers for each engine  
  cpp/            # native C++ implementation (pybind11)  
  rust/           # native Rust implementation (PyO3)  
  runner.py       # BenchmarkRunner  
  db.py           # BenchmarkDB (SQLite)  
  api/            # Flask REST API  
frontend/         # React + TypeScript + Vite  
experiments/      # CLI experiment drivers  
results/          # SQLite database, generated tables  
tests/            # pytest suite
```

The Python package is pure-Python apart from optional native extensions. The C++ extension is built with `pip install -e benchmarking/cpp/` (using a `setup.py` that invokes `pybind11`); the Rust extension is built with `maturin develop -release` inside `benchmarking/rust/`. If either extension is not built, the corresponding engine is silently excluded from the engine registry at runtime, so the framework remains fully functional on systems without a C++ or Rust toolchain.

4.2 Core Modules

4.2.1 Workload Registry

The workload registry is implemented in `benchmarking/workloads/base.py`. The decorator `@workload(name)` performs three actions in a single pass: it applies `dataclass` to the decorated class, sets a class-level attribute `workload_type = name`, and registers the resulting class in the module-level dictionary `WORKLOAD_REGISTRY`. The base class `WorkloadConfig` provides concrete implementations of `validate()`, `to_dict()`, `from_dict()` and `config_hash()` so that no subclass is required to override anything except its field list and (optionally) its validation logic.

The free function `config_from_dict(d)` reads `d["workload_type"]`, looks up the corresponding class in `WORKLOAD_REGISTRY`, and delegates to that class's `from_dict()`. Unknown keys are silently dropped during deserialisation, so configurations stored by older versions of the framework continue to load even after new fields are added. `config_hash()` serialises the configuration with `json.dumps(..., sort_keys=True)` and returns the first eight characters of the SHA-256 digest as a stable, machine-independent fingerprint.

4.2.2 Monte Carlo Engine Interface

All engines are subclasses of the abstract base class `MonteCarloEngine` defined in `benchmarking/engines/base.py`:

```
class MonteCarloEngine:
    name: str
    language: str
    backend: str

    def supports(self, workload_type: str) -> bool: ...
    def supported_ad_modes(self) -> tuple[str, ...]: ...
    def run(self, config: WorkloadConfig,
            ad_mode: str = "none") -> tuple[float, dict | None]: ...
```

The class attributes `name`, `language` and `backend` are the values written to the corresponding columns of `benchmarks.db`, enabling later queries such as “all Rust runs” or “all GPU runs”. The `run()` method is the only computational entry point: it accepts a fully-validated `WorkloadConfig` and an AD mode string, and returns (price, greeks). The `greeks` component is either `None` (for `ad_mode = "none"`) or a dictionary mapping Greek names (`"delta"`, `"vega"`, `"rho"`, ...) to floats.

4.2.3 NumPy Engine

`CPUMonteCarloEngine` (`benchmarking/engines/mc_cpu.py`) is the single-threaded NumPy reference implementation. For the European workload it draws all M standard normals in one call to `np.random.default_rng(seed).standard_normal(M)`, evaluates the closed form for S_T vectorially and computes the discounted mean payoff. For the local-volatility workload it runs the N -step log-Euler loop in Python, with each iteration operating on a NumPy array of M paths and using the stable `softplus`

```
def softplus(z):
    return np.maximum(z, 0.0) + np.log1p(np.exp(-np.abs(z)))
```

The same module exposes Black–Scholes closed-form helpers for price, Delta, Vega and Rho, which the runner calls to populate the analytical reference columns of every database row.

4.2.4 JAX Engine

`JAXMonteCarloEngine` (`benchmarking/engines/mc_jax.py`) is the only engine that implements AD. The European kernel is declared at module level to maximise compilation cache hits:

```
@functools.partial(jax.jit, static_argnames=("option_type",))
def _european_price(S0, r, sigma, K, T, Z, option_type):
    sqrtT = jnp.sqrt(T)
    ST = S0 * jnp.exp((r - 0.5 * sigma**2) * T + sigma * sqrtT * Z)
    payoff = jnp.where(option_type == "call",
                       jnp.maximum(ST - K, 0.0),
                       jnp.maximum(K - ST, 0.0))
    return jnp.exp(-r * T) * jnp.mean(payoff)
```

Reverse-mode Greeks for the European workload are obtained from a single backward pass:

```
grad_fn = jax.jit(jax.grad(_european_price, argnums=(0, 1, 2)),
                  static_argnames=("option_type",))
delta, rho, vega = grad_fn(S0, r, sigma, K, T, Z, option_type)
```

Forward-mode uses three `jax.jvp` calls, one per input direction, each probing the corresponding basis tangent vector. The local-volatility kernel expresses the N -step loop as `jax.lax.scan`:

```

def _step(carry, z):
    S, t = carry
    x = jnp.log(S / S0)
    raw = a0 + a1 * x + a2 * x * x + b1 * t
    sigma = sigma_min + softplus(raw)
    S_next = S * jnp.exp((r - 0.5 * sigma**2) * dt
                        + sigma * jnp.sqrt(dt) * z)
    return (S_next, t + dt), None

(S_T, _) = jax.lax.scan(_step, (S0_arr, 0.0), Z_matrix)

```

`jax.lax.scan` produces a single loop in the XLA graph; tracing time and peak memory remain constant in N , in contrast with a Python `for` loop which would unroll the graph to size $\Theta(N)$ and substantially increase compile time and memory for $N = 252$.

The AD overhead is computed by the runner as the ratio of the mean timed runtime of the AD run to that of an identically-configured no-AD baseline run. Crucially, both legs use the same warmup count, so XLA cache state is equivalent and the ratio reflects only the differentiation cost.

4.2.5 C++ Engine

The native implementation lives in `benchmarking/cpp/cpu_engine.cpp / cpu_engine.hpp` and is exposed to Python via `pybind11` bindings in `benchmarking/cpp/bindings/pybind_module.cpp`. The Python wrapper `benchmarking/engines/mc_cpp.py` loads the compiled module lazily and falls back to a not-available stub if the import fails.

Parallelisation is via OpenMP. Each thread owns its own `std::mt19937_64` generator, seeded as `seed + thread_id` to guarantee statistical independence across threads while remaining reproducible for any fixed thread count. The European inner loop is:

```

double sum = 0.0;
#pragma omp parallel reduction(+:sum)
{
    int tid = omp_get_thread_num();
    std::mt19937_64 rng(seed + tid);
    std::normal_distribution<double> N01(0.0, 1.0);
    #pragma omp for
    for (int i = 0; i < M; ++i) {
        double Z = N01(rng);
        double ST = S0 * std::exp((r - 0.5 * sigma * sigma) * T
                                + sigma * std::sqrt(T) * Z);
        sum += std::max(option_type == 0 ? ST - K : K - ST, 0.0);
    }
}
double price = std::exp(-r * T) * sum / M;

```

The local-volatility kernel reuses the same RNG strategy and implements the log-Euler loop with the numerically stable `softplus` (`raw > 0 ? raw : 0`) + `std::log1p(std::exp(-std::abs(raw)))`, identical to the Python and Rust formulations.

4.2.6 Rust Engine

The native implementation lives in `benchmarking/rust/src/lib.rs` and is exposed to Python via `PyO3` bindings, built with `maturin develop -release`. The Python wrapper `benchmarking/engines/mc_rust.py` mirrors the C++ wrapper: lazy import, graceful fallback, identical engine interface.

The Rust implementation uses `SmallRng` (Xoshiro128++) from the `rand` crate for high-throughput sampling, with each Rayon worker thread seeded as `seed XOR thread_index`. The simulation is expressed as a data-parallel iterator over the path range:

```

let price: f64 = (0..M).into_par_iter()
    .map_init(
        || SmallRng::seed_from_u64(seed ^ rayon::current_thread_index())
    )

```

```

                                .unwrap_or(0) as u64),
    |rng, _| {
        let z: f64 = StandardNormal.sample(rng);
        let s_t = s0 * ((r - 0.5 * sigma * sigma) * t
                        + sigma * t.sqrt() * z).exp();
        (s_t - k).max(0.0)
    })
    .sum:<f64>() * (-r * t).exp() / (M as f64);

```

The softplus is implemented inline as `z.max(0.0) + (-z.abs()).exp().ln_1p()`, matching the Python, C++ and JAX formulations bit-for-bit.

4.2.7 CUDA Engine

`CUDAMonteCarloEngine` (`benchmarking/engines/mc_cuda.py`) uses PyCUDA to compile and launch CUDA kernels at runtime. At import time, the engine runs a minimal 128-path trial; if CUDA initialisation, kernel compilation, or the trial run fails, the engine sets its availability flag to `False` and is excluded from `/api/engines`.

The European kernel uses one thread per Monte Carlo path:

```

__global__ void price_kernel(double S0, double r, double sigma,
                             double K, double T, double sqrtT,
                             unsigned long long seed,
                             int option_type, int M,
                             double *payoffs) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx >= M) return;
    curandState state;
    curand_init(seed, idx, 0, &state);
    double Z = curand_normal_double(&state);
    double ST = S0 * exp((r - 0.5 * sigma * sigma) * T + sigma * sqrtT * Z);
    double payoff = option_type == 0
        ? fmax(ST - K, 0.0) : fmax(K - ST, 0.0);
    payoffs[idx] = exp(-r * T) * payoff;
}

```

Using the thread index as the `curand_init` sequence guarantees statistical independence regardless of block or grid layout. The host averages the resulting payoff array with `np.mean`.

Pathwise forward-mode Delta is implemented in a second kernel. Using the identity $\partial S_T / \partial S_0 = S_T / S_0$ for GBM, the pathwise Delta integrand is

$$\mathbf{1}_{\{S_T > K\}} \cdot \frac{S_T}{S_0} \quad (\text{call}), \quad -\mathbf{1}_{\{S_T < K\}} \cdot \frac{S_T}{S_0} \quad (\text{put}),$$

and the engine returns $\Delta = e^{-rT} \mathbb{E}[d(\text{payoff})/dS_0]$. Note that this is a *pathwise* estimator, not a finite-difference approximation; it produces unbiased Greeks with the same variance characteristics as a single-pass forward-mode AD.

4.2.8 Benchmark Runner

`BenchmarkRunner` (`benchmarking/runner.py`) is a small class. Construction takes an engine instance, an optional display name and the warmup and repetition counts. The `run()` method implements the protocol of Section 3.5. Timed repetitions use `time.perf_counter()` which provides nanosecond resolution on Linux and macOS. The runner separately computes the analytical reference values (by calling the Black–Scholes helpers in `mc_cpu.py`) and the absolute and relative errors for the price and each Greek, so that the database row produced is self-contained.

When `ad_mode != "none"` the runner first performs the AD timed run, then a matched no-AD baseline run using the same engine, same configuration, same warmup count and same repetition count. This ensures that the AD overhead ratio reflects only the cost of differentiation, not the cost of XLA recompilation or any other one-time setup.

4.2.9 Metrics Collection

The runner collects four classes of metrics:

- **Timing:** mean, standard deviation, min, max of the timed repetitions, in milliseconds, plus the derived `throughput_paths_per_sec = M / mean_runtime_s`.
- **AD overhead:** the matched baseline mean runtime (`baseline_mean_ms`) and the overhead ratio $\bar{t}_{AD}/\bar{t}_{baseline}$ (`ad_overhead_ratio`).
- **Memory:** process RSS sampled before and after the timed repetitions via `psutil`; the recorded value is the difference.
- **Numerical accuracy:** absolute and relative error for the price and for each Greek against the Black–Scholes reference, when applicable.

Environment metadata is captured once per `run()` call: the wall-clock timestamp, the Python version and implementation, the platform string, the CPU model and architecture, the logical CPU count, total RAM in GB, the NumPy version and the JAX version.

4.2.10 Storage Layer

BenchmarkDB (`benchmarking/db.py`) is a thin SQLite wrapper. A new connection is opened for each call with `check_same_thread=False`, and an explicit `threading.Lock` serialises write operations. The database is created in WAL mode the first time it is opened. The schema is described in detail in Section 3.6; it is defined declaratively in the `_FULL_SCHEMA_COLUMNS` constant, and any missing columns are added automatically at startup via `PRAGMA table_info` followed by `ALTER TABLE`.

Two write paths coexist. Experiment scripts use `store_run_full()`, which writes a fully-populated `completed` row in a single transaction. The REST API uses the three-step lifecycle `create_run()` → `mark_running()` → `mark_completed()` (or `mark_failed()`) to support asynchronous execution from a background worker thread.

4.2.11 REST API

The Flask application (`benchmarking/api/server.py`) registers the seven endpoints listed in Table 3.3 and starts a single daemon background thread on startup. The worker polls the `runs` table for `pending` rows in FIFO order, dispatches each through the appropriate engine via `BenchmarkRunner.run()`, and updates the row with the result. Engine availability is probed once at startup: C++ and CUDA engines that fail to load or fail their probe runs are removed from the registry.

The API returns JSON in all cases and uses standard HTTP status codes. Flask-CORS is configured to allow requests from the Vite development server (port 5173) so that the frontend can be developed locally without a reverse proxy.

4.2.12 Frontend / Dashboard

The frontend is a React + TypeScript single-page application built with Vite and styled with Material UI. The directory layout follows the conventional `src/pages/`, `src/components/` and `src/api/` structure. A typed HTTP client in `src/api/client.ts` wraps every API endpoint with a TypeScript function whose return type mirrors the corresponding Python dataclass.

Pages:

- **SimulatePage** — submits a new run via `POST /api/runs` and polls `GET /api/runs/:id` until the status becomes `completed` or `failed`.
- **ComparePage** — consumes `/api/compare_matrix` to render a multi-engine, multi-AD-mode comparison grid.
- **HistoryPage** — filterable table backed by `GET /api/runs`.
- **SummaryPage** — consumes `/api/summary`.
- **AnalysisPage** — focused view of AD overhead and Greek accuracy.

Components are factored to be workload-agnostic: `SimulationForm` is generated dynamically from the workload `SCHEMA` returned by the API, so adding a new workload class (or even a new field) requires *no* frontend changes.

4.3 Supported Workloads

Three workloads are implemented and validated end-to-end:

- **european** (`EuropeanOptionConfig`): single-step exact GBM. Supported by all five engines. Used for AD overhead, Greek accuracy, language comparison and throughput scaling experiments.
- **european_local_vol** (`EuropeanLocalVolConfig`): N -step log-Euler simulation of GBM with a parametric local-volatility surface. Supported by NumPy, JAX, C++ and Rust; CUDA support is not implemented in the current iteration. Used to stress-test AD on a deeper computation graph and to expose the cost of multi-step over single-step simulation.
- **asian** (`AsianOptionConfig`): arithmetic-average path-dependent option workload. Supported by the Python and JAX engines in the final cloud-profiler experiments. Used to check that back-end rankings and AD overheads are not inferred only from terminal-payoff European workloads.

4.4 AD Integration

Automatic differentiation is implemented exclusively in the JAX and CUDA engines. The JAX engine supports both forward and reverse mode for both workloads; the CUDA engine implements only forward-mode pathwise Delta on the European workload.

For the European workload the AD output is a three-tuple $(\Delta, \rho, \text{Vega}) = (\partial P / \partial S_0, \partial P / \partial r, \partial P / \partial \sigma)$. Forward mode is implemented as three `jax.jvp` calls (one per input direction), and reverse mode as a single `jax.grad` call with `argnums=(0,1,2)`.

For the local-volatility workload the AD output is a six-tuple $(\partial P / \partial S_0, \partial P / \partial a_0, \partial P / \partial a_1, \partial P / \partial a_2, \partial P / \partial b_1, \partial P / \partial \sigma)$. Forward mode therefore requires six `jax.jvp` calls, while reverse mode again computes the full gradient in a single backward pass.

Both modes feed into the runner’s overhead-ratio computation. Because the gradient and the baseline price share the same forward computation, the overhead ratio is a clean measure of the marginal cost of differentiation.

4.5 GPU Support

CUDA support is implemented for the European workload via PyCUDA. The kernel launch configuration is determined at runtime: the host queries the device for the maximum number of threads per block and chooses the largest power of two not exceeding it, then computes the number of blocks as $\lceil M / \text{threads_per_block} \rceil$. The host allocates a single device array of size M for the payoffs and reduces it to a scalar with `np.mean` after copying back. The CUDA engine is probed at server startup with a 128-path trial run; if the probe fails (no GPU, no CUDA toolkit, PyCUDA import failure, etc.) the engine is silently excluded.

JAX inherits GPU support transparently when JAX is installed with CUDA wheels. The same JAX engine code paths execute on GPU without modification; the benchmark database simply records a different platform string. Switching between CPU and GPU JAX is therefore a deployment decision rather than a code change.

4.6 Challenges

Several issues arose during implementation and shaped the final design.

XLA caching across runs. A naive initial JAX implementation defined the kernel inside the engine’s `run()` method. Each call created a fresh function object, defeating the JIT cache: every timed repetition paid the full ≈ 115 ms compile cost. Moving the kernel to module level reduced warm execution time to ≈ 0.6 ms and made the JAX engine genuinely competitive on small M .

Triple AD passes. An earlier reverse-mode implementation invoked `jax.grad` three times with `argnums=0,1,2`. The result was numerically correct but tripled both the measured AD wall time and the AD overhead ratio. Consolidating into a single `jax.grad(..., argnums=(0,1,2))` call eliminated this artefact; this single change halved the reverse-mode overhead in early measurements and brought the framework into line with how reverse-mode AD is deployed in practice.

for-loop unrolling in JAX. The first local-volatility implementation used a Python `for` loop over the $N = 252$ Euler steps. Compilation time grew linearly in N and peak memory reached several gigabytes during tracing. Rewriting the loop as `jax.lax.scan` produced a single fused loop in the XLA graph; compile time and memory became effectively independent of N .

Numerically stable softplus. A naive softplus implementation $\log(1+e^z)$ overflows for $z \gtrsim 700$ in double precision and loses precision near $z = 0$. The stable form $\max(z, 0) + \log(1+e^{-|z|})$ is used identically in NumPy, JAX, C++ and Rust, so any cross-engine numerical discrepancy can only be attributed to the RNG or the floating-point order of operations, not to the activation function itself.

Per-thread RNG reproducibility. Both the C++ and Rust engines use per-thread RNG streams seeded from the global seed plus the thread index. Changing the thread count therefore changes the RNG assignment and produces statistically equivalent but numerically different results. This is recorded by storing `num_threads` on every run, so that runs at different thread counts are treated as distinct configurations.

Optional native components. Both the C++ and Rust engines require non-trivial native toolchains (a C++17 compiler with OpenMP, or the Rust toolchain plus maturin) that are not present in every environment. The engines are loaded lazily and degrade gracefully: a missing build is reported only when the user explicitly requests that engine via the API, and the corresponding rows simply do not appear in the engine list.

Schema evolution. During development, several new columns were added to the database (notably the AD-overhead, error-metric and cloud columns). Rather than write migration scripts, the database opens with a self-migrating `_init_schema()` that compares the existing columns against the authoritative list and `ALTER TABLE`s any missing columns into existence. This has allowed every historical database file to remain readable across schema changes.

4.7 Cloud Metadata and Cost Support

Cloud support is implemented as metadata attached to ordinary benchmark rows rather than as a separate execution path. Experiment drivers can record the cloud provider, instance type, region, zone, hourly price assumption, git commit, and derived cost per run. This preserves the same framework invariants as local execution: every cloud result still has a workload configuration, an engine, an AD mode, timing statistics, numerical outputs, and environment metadata.

The final cloud experiment uses Google Cloud instance metadata recorded in the result files, including `n2-standard-4` in `eu-west1-b`. Cost per run is derived from measured runtime and the stored hourly price. The implementation deliberately avoids hard-coding a claim that one instance is universally cheapest: prices and availability can change, so the report treats cost as experiment metadata associated with a particular run.

4.8 Profiler Implementation

The final profiler-versus-grid experiment is driven by `experiments/run_profiler_vs_grid.py`. The script constructs the candidate grid, filters unsupported workload/engine/AD combinations, executes probe runs, applies the old profiler and SHA selection policies, and records which configurations would be promoted to full benchmarking. The report artefact generator `Final_Year_Project_Report/evaluation/generate_report_assets.py` then converts the stored results into the tables and figures used in Chapter 6.

In the final `sha_cloud_profiler_v4` experiment, the full-grid baseline contains 16 candidate configurations evaluated at three full path counts, for 48 full benchmark cells. The old probe-based profiler selects 30 full cells, while SHA selects 24. The SHA implementation records intermediate promotion rounds, including the active configuration count at each probe path budget. This metadata is used to generate the SHA progression plot and to compute probe-budget savings.

The profiler implementation also computes decision-quality metrics against the full grid: Pareto-frontier recovery, runtime regret, cost regret, and Spearman rank correlation between probe and full-scale rankings. This is important because the profiler is an optimisation of the benchmarking process, not a replacement for evaluation. Its success is judged by whether it reduces expensive full runs while preserving the decisions that the full grid would have supported.

4.9 Testing

The framework ships with 55 tests across two pytest files in `tests/`.

`test_system.py` covers:

- Six parametric tests for configuration validation (invalid field values, rejected `option_type`).
- Configuration serialisation: round-trip `to_dict` / `from_dict`, exclusion of `SCHEMA` from the serialised form, `config_from_dict` dispatch, and the expected `ValueError` on an unknown `workload_type`.
- Engine correctness: NumPy and JAX pricing accuracy against Black-Scholes within tolerance.
- Runner statistics: structure of the timing record and reproducibility under fixed seed.
- Database round-trips: `create_run`, `mark_running`, `mark_completed`, `get_run`, `get_all_runs`, `summary`.
- API endpoint contracts: status codes and response structure for all seven endpoints.
- An end-to-end flow from configuration through engine and runner to database insert and API read-back.

`test_ad_framework.py` covers:

- JAX engine executes without error for each of `ad_mode` $\in$ `{none, forward, reverse}` on the European workload.
- The standalone differentiated price function exposes the expected JAX function signature.
- Analytical Greek computations agree with their closed-form definitions.
- Computed gradients fall within the expected analytical bounds.

4.10 Dependencies

Core Python dependencies: `numpy`>=1.26, `scipy`>=1.11, `jax[cpu]`>=0.4.28, `flask`>=3.0, `flask-cors`>=4.0, `pytest`>=8.0, `pybind11`>=2.11 (for the C++ build). `psutil` is optional and enables RSS-based memory profiling when present. The Rust engine requires the Rust toolchain (`rustup`), `maturin`>=1.0, `PyO3`, and the `rand`, `rand_distr` and `rayon` crates. The CUDA engine requires the NVIDIA CUDA toolkit and `PyCUDA`. The frontend depends on `react`, `react-dom`, `@mui/material` and a Vite-based toolchain.

4.11 Summary

The implementation realises the design of Chapter 3 in roughly 4,000 lines of Python plus native C++ and Rust extensions and a TypeScript frontend. Five engines (NumPy, JAX, C++, Rust, CUDA), two workloads (European GBM, European local volatility) and three AD modes (none, forward, reverse) are supported, with full SQLite persistence, a REST API, a React dashboard and a 55-test pytest suite. The next chapter describes the experimental protocol applied to this implementation.

Chapter 5

Experimental Setup

5.1 Overview

This chapter defines the experimental methodology used to evaluate the Monte Carlo benchmarking framework. The aim is to ensure that all measurements reported in Chapter 6 are produced from controlled, reproducible, and scientifically interpretable experiments.

The evaluation combines two forms of evidence. First, a detailed local CPU campaign is performed on a GitHub Codespace with 4 CPU cores and 16 GB RAM. This campaign supports the detailed validation, scalability, AD-overhead, thread-scaling, memory-pressure, and workload-sensitivity analysis. Second, a final cloud campaign evaluates cost-performance and profiler quality on Google Cloud instances, with the headline `sha_cloud_profiler_v4` run executed on `n2-standard-4` in `eu-west1-b`. GPU execution, distributed execution, mixed precision, and Mojo-based vendor-claim validation remain future extensions rather than completed experiments.

The experimental design follows three principles.

1. **Like-for-like workloads:** every engine is evaluated on the same workload configuration, random seed policy, path count, model parameters, and timing procedure.
2. **Separation of concerns:** performance differences are isolated by varying one main factor at a time, such as engine, AD mode, path count, time-step count, or requested thread count.
3. **Auditable measurement:** each benchmark run stores timing statistics, numerical outputs, memory measurements, software versions, hardware metadata, and the full workload configuration in the shared SQLite database.

Three workloads are used across the complete evaluation. The European GBM workload is used as a simple analytical validation case and low-cost baseline. The parametric local-volatility workload is used as the main local performance workload because it contains time stepping, a nonlinear volatility surface, and a larger computational graph for automatic differentiation. The Asian option is used in the final cloud-profiler experiments as a path-dependent workload that exposes a different backend ranking from the two European-style workloads.

5.2 Connection to the Research Questions

Table 5.1 maps the project research questions to the experiments that generate the evidence used in the results chapter. Cloud hardware is evaluated through the final cost-performance and profiler experiments. GPU acceleration, distributed execution, and vendor-specific Mojo claims are not directly evaluated; they are discussed as scope limitations and future work.

5.3 Execution Environment

The detailed local experiments are executed in a GitHub Codespace with 4 CPU cores and 16 GB RAM. The exact CPU model, operating system, Python version, NumPy version, JAX version, and other software metadata are captured automatically in the benchmark database for every run.

Table 5.1: Mapping from research questions to local experiment evidence.

Research question	Evidence collected	Script(s)
How do software stacks affect MC+AD throughput and efficiency?	Runtime, throughput, numerical output, and memory measurements are compared across NumPy, JAX/XLA, C++/OpenMP, and Rust/Rayon under the same workload definitions.	<code>run_european_baseline.py</code> ; <code>run_european_scalability.py</code> ; <code>run_localvol_suite.py</code>
What are the trade-offs between programming languages and AD libraries?	No-AD runtime and throughput are compared across Python/NumPy, JAX/XLA, C++/OpenMP, and Rust/Rayon. Forward- and reverse-mode AD are evaluated through JAX, which provides the AD-capable reference implementation.	<code>run_european_baseline.py</code> ; <code>run_european_ad_analysis.py</code> ; <code>run_localvol_suite.py</code>
How do forward- and reverse-mode AD affect MC workloads?	AD overhead is measured by comparing JAX no-AD, forward-mode AD, and reverse-mode AD runs at matched path counts, time-step counts, warmup counts, and repetition counts.	<code>run_european_ad_analysis.py</code> ; <code>run_localvol_suite.py</code>
How do path count and time-step count affect scalability?	The experiments sweep the number of Monte Carlo paths M and, for the local-volatility workload, the number of time steps N . This separates path-level scaling from discretisation-cost scaling.	<code>run_european_scalability.py</code> ; <code>run_localvol_suite.py</code>
How do parallelisation strategies affect scaling and bottlenecks?	Strong and weak thread-scaling experiments vary the requested CPU thread count over 1, 2, 4 threads. Observed thread counts are stored where available to distinguish requested from actual runtime behaviour.	<code>run_thread_scalability.py</code> ; <code>run_localvol_suite.py</code>
Which Google Cloud configurations provide good cost/performance?	Runtime and estimated cost per run are compared for priced cloud instances, using stored instance metadata and generated report tables.	<code>run_profiler_vs_grid.py</code> ; <code>generate_report_assets.py</code>
Can profiling reduce full cloud benchmarking cost?	The old profiler and SHA are compared with a full-grid baseline using full runs saved, Pareto recovery, runtime regret, cost regret, and rank correlation.	<code>run_profiler_vs_grid.py</code>
How far can the framework support broader proposal goals?	The framework records enough configuration and environment metadata to support cloud, GPU, distributed, mixed-precision, BLAS, or Mojo experiments, although not all of these extensions are completed in the final evaluation.	All experiment scripts and database records

The use of a single local environment is deliberate: it reduces variation from uncontrolled hardware changes and allows the local experiments to focus on the framework, workload definitions, engines, AD modes, and CPU scaling behaviour. Cloud conclusions are drawn only from the later cloud-profiler experiments and are tied to their recorded instance types, regions, pricing assumptions, and software environment.

5.4 Workloads

5.4.1 European Option under Geometric Brownian Motion

The validation workload prices a European call option under geometric Brownian motion (GBM). Under the risk-neutral measure, the terminal asset price is

$$S_T = S_0 \exp \left[\left(r - \frac{1}{2} \sigma^2 \right) T + \sigma \sqrt{T} Z \right], \quad Z \sim \mathcal{N}(0, 1).$$

The Monte Carlo estimator for the discounted call price is

$$\hat{P} = e^{-rT} \frac{1}{M} \sum_{i=1}^M \max(S_T^{(i)} - K, 0),$$

where M is the number of simulated paths. Unless otherwise stated, the fixed parameters are

$$S_0 = 100, \quad K = 100, \quad r = 0.05, \quad \sigma = 0.20, \quad T = 1.0, \quad \text{seed} = 42.$$

This workload is used for analytical validation, simple baseline performance, path-count scalability, and JAX AD correctness. Its main advantage is that the Black–Scholes analytical price and Greeks provide a direct reference for the Monte Carlo price, Delta, Vega, and Rho.

5.4.2 European Option under Parametric Local Volatility

The main performance workload prices the same European payoff, but replaces constant volatility with a parametric local-volatility surface. The asset path is advanced using a log-Euler discretisation over N time steps:

$$S_{t+\Delta t} = S_t \exp \left[\left(r - \frac{1}{2} \sigma(S_t, t)^2 \right) \Delta t + \sigma(S_t, t) \sqrt{\Delta t} Z_t \right], \quad Z_t \sim \mathcal{N}(0, 1).$$

The local volatility surface is parameterised as

$$\sigma(S, t) = \sigma_{\min} + \text{softplus} \left(a_0 + a_1 \log(S/S_0) + a_2 \log(S/S_0)^2 + b_1 t \right).$$

The softplus function is

$$\text{softplus}(x) = \log(1 + e^x).$$

In implementation, softplus is evaluated using the numerically stable form

$$\text{softplus}(x) = \max(x, 0) + \log(1 + e^{-|x|}),$$

which reduces overflow risk for large positive inputs.

The default local-volatility setting uses daily time steps,

$$N = 252, \quad \sigma_{\min} = 0.01, \quad a_1 = -0.10, \quad a_2 = 0.20, \quad b_1 = 0.00.$$

The parameter a_0 is chosen so that the at-the-money time-zero volatility equals 0.20:

$$a_0 = \log(\exp(0.20 - \sigma_{\min}) - 1).$$

This local-volatility workload is more demanding than the single-step GBM case because each path requires repeated state updates and repeated evaluation of a nonlinear volatility function. It is therefore used as the main workload for engine comparison, AD overhead analysis, path-count scaling, time-step scaling, thread scaling, surface-shape sensitivity, and local resource-usage analysis.

5.4.3 Asian Arithmetic-Average Option

The final cloud-profiler experiments also include an Asian call option whose payoff depends on the arithmetic average of the simulated asset path. In simplified form, the discounted payoff estimator is

$$\hat{P}_{\text{Asian}} = e^{-rT} \frac{1}{M} \sum_{i=1}^M \max \left(\frac{1}{N} \sum_{j=1}^N S_{t_j}^{(i)} - K, 0 \right).$$

This workload is included because it is path dependent: the payoff depends on the trajectory rather than only on the terminal value. It therefore introduces a different memory-access and compilation pattern from the European GBM baseline. In the final cloud comparison, the Asian workload is evaluated for supported Python/JAX configurations and contributes to the AD-overhead, cost-performance, and profiler-selection results.

5.5 Engines and Software Stacks

Table 5.2 summarises the engine configurations used in the final experiments. Engines that are not available in a particular run are skipped and recorded as unavailable rather than silently excluded.

Table 5.2: Engine configurations used in the local experimental programme.

Engine label	Language	Backend	Role in evaluation
<code>cpu</code>	Python	NumPy / CPU	Reference vectorised Python implementation. Used as a baseline software stack for no-AD CPU throughput and correctness checks.
<code>jax</code>	Python	JAX / XLA	Compiled AD-capable implementation. Used for no-AD performance, forward-mode AD, reverse-mode AD, and XLA compiled execution behaviour.
<code>cpp</code>	C++	OpenMP	Compiled CPU implementation. Used for low-level CPU throughput and thread-scaling comparison when the extension module is built successfully.
<code>rust</code>	Rust	Rayon	Compiled CPU implementation. Used for language/runtime comparison and parallel CPU throughput on the local-volatility workload.

The exact Python, NumPy, JAX, compiler, operating-system, CPU, memory, and available backend metadata are captured per run and stored in the database. This avoids relying on manually written machine descriptions, which can become incomplete or stale.

5.6 Experimental Variables

The experiments are designed around a small set of controlled variables.

5.6.1 Independent Variables

The main independent variables are

- **Engine:** NumPy CPU, JAX/XLA, C++/OpenMP, and Rust/Rayon.
- **Path count:** the number of Monte Carlo paths, denoted by M .

- **Time steps:** the number of simulated time steps, denoted by N . For the exact GBM workload, $N = 1$ implicitly because the terminal distribution is sampled directly. For local volatility, N is explicitly swept in selected experiments.
- **AD mode:** no AD, forward-mode AD, or reverse-mode AD. Forward and reverse modes are evaluated through the JAX engine.
- **Thread count:** requested CPU worker count for strong and weak thread-scaling experiments.
- **Volatility-surface preset:** local-volatility surface shape, such as flat, equity-skew, or strong-smile, where supported by the experiment runner.
- **Cloud instance:** Google Cloud instance type and region for final cost-performance and profiler experiments.
- **Profiler policy:** full grid, old probe-based profiler, or successive halving.

5.6.2 Dependent Variables

The main dependent variables are

- mean runtime in milliseconds;
- runtime standard deviation, minimum, and maximum;
- throughput in paths per second;
- peak process memory usage in megabytes;
- Monte Carlo price estimate;
- Greek estimates for AD-enabled GBM experiments;
- absolute and relative error against analytical references where available;
- AD overhead ratio;
- requested and observed process thread counts where available;
- coefficient of variation for selected stability experiments.

5.6.3 Controlled Variables

Unless explicitly swept, the following are held fixed:

- option parameters S_0 , K , r , σ , and T ;
- random seed;
- workload definition;
- local-volatility surface parameters;
- warmup count and timed repetition count within each experiment;
- database schema and result-writing path;
- timing method;
- correctness thresholds used for validation.

5.7 Metrics

For each benchmark cell, the script performs a configurable number of warmup runs followed by a configurable number of timed runs. Warmup timings are discarded. The timed runs produce

$$\bar{t}_{\text{ms}} = \frac{1}{R} \sum_{j=1}^R t_j,$$

where R is the number of timed repetitions and t_j is the runtime of the j th repetition in milliseconds.

Throughput is reported as

$$\text{throughput} = \frac{M}{\bar{t}_{\text{ms}} \times 10^{-3}} \quad \text{paths per second.}$$

For AD experiments, the overhead ratio is computed against a matched no-AD baseline:

$$\text{AD overhead} = \frac{\bar{t}_{\text{AD}}}{\bar{t}_{\text{none}}},$$

where both runs use the same engine, workload, path count, time-step count, warmup count, and number of timed repetitions.

For the GBM workload, numerical error is measured against the analytical Black-Scholes reference:

$$\text{relative error} = \frac{|x_{\text{MC}} - x_{\text{ref}}|}{\max(|x_{\text{ref}}|, \epsilon)},$$

where x may be the price, Delta, Vega, or Rho, and ϵ is a small constant used only to avoid division by zero.

For the local-volatility workload, no closed-form analytical reference is available for the chosen parametric surface. Cross-engine numerical consistency is therefore assessed by comparing estimates across engines and by checking convergence as M increases. Where available, the Monte Carlo standard error is used to interpret whether differences between engines are consistent with sampling variation.

For selected stability experiments, the coefficient of variation is reported as

$$\text{CV} = \frac{s_t}{\bar{t}_{\text{ms}}},$$

where s_t is the sample standard deviation of the timed repetitions.

5.8 Measurement Methodology

5.8.1 Warmup and JIT Compilation

Each script performs warmup repetitions before measurement. This is especially important for JAX/XLA, where the first invocation may include tracing and compilation. The reported runtime is the steady-state execution time after warmup, not the cold-start compilation time.

Cold-start costs are not ignored entirely. Where relevant, they are discussed separately in the results chapter because they affect short-lived interactive workloads differently from repeated production-style simulation runs.

5.8.2 Repeated Measurements

Each benchmark cell is measured over multiple timed repetitions. The database stores the mean, standard deviation, minimum, and maximum runtime rather than a single timing. This makes it possible to identify unstable measurements and avoids drawing conclusions from one-off timing noise.

Most main experiments use 7 timed runs and 3 warmup runs. Smoke tests use 1 timed run and 1 warmup run and are not treated as reported performance results. Selected stability experiments use a larger number of timed runs to estimate measurement variability.

5.8.3 Randomness and Correctness

All workloads use an explicit seed. Identical seeds improve reproducibility within each engine, but bit-identical output is not required across engines because different engines may use different random-number generators. Cross-engine correctness is therefore assessed using analytical references where available, and by checking that estimates converge within reasonable Monte Carlo error as M increases.

For the European GBM workload, correctness is assessed directly against the Black–Scholes analytical price and Greeks. For local volatility, the emphasis is on cross-engine consistency and convergence with increasing path count, since the chosen parametric local-volatility model does not have the same closed-form reference.

5.8.4 Fairness Controls

The following controls are applied across experiments:

- Engines receive the same workload parameters and path counts.
- AD overhead is computed only against matched no-AD runs.
- JAX timings are synchronised so that asynchronous execution does not under-report runtime.
- Thread-scaling cells are run in isolated subprocesses where possible to reduce contamination from persistent thread-pool state.
- Failed, skipped, or unavailable engines are recorded explicitly rather than removed silently from the analysis.
- Smoke-test results are used only to validate execution and are not interpreted as performance results.

5.8.5 Data Exclusion Policy

Runs are excluded from the final comparative analysis only under predefined conditions:

- the run terminates with an exception;
- the detected engine or backend does not match the intended experiment cell;
- the result violates the experiment’s correctness threshold;
- required metadata such as engine, workload, path count, time-step count, or runtime is missing;
- the machine is known to have been under unrelated heavy load during the run.

Excluded runs are not deleted from the database. They remain available for audit, but are filtered out of the final completed-run queries.

5.9 Experiment Scripts

Table 5.3 summarises the core experiment scripts used in the final evaluation.

5.10 Individual Experiment Designs

5.10.1 European GBM Baseline

The European GBM baseline establishes a simple no-AD performance reference and a correctness check against the Black–Scholes analytical price.

Script. `experiments/run_european_baseline.py`

Workload. European GBM with fixed option parameters.

Engines. NumPy CPU, JAX/XLA, C++/OpenMP, and Rust/Rayon where available.

Purpose. This experiment provides a low-cost first comparison across engines and checks that the framework can run each implementation through the same configuration and database-writing path.

Table 5.3: Core experiment scripts and their roles.

Script	Experiment type or labels	Purpose
<code>run_european_baseline.py</code>	<code>european_baseline</code>	Compares no-AD throughput across available engines on the single-step European GBM workload.
<code>run_european_ad_analysis.py</code>	<code>european_ad_analysis</code>	Measures JAX forward- and reverse-mode AD overhead and validates Greek accuracy against Black–Scholes references.
<code>run_european_scalability.py</code>	<code>european_scalability</code>	Sweeps path count M to measure how runtime and throughput scale on the simple GBM workload.
<code>run_thread_scalability.py</code>	<code>thread_scalability_strong</code> ; <code>thread_scalability_weak</code>	Measures strong and weak CPU thread scaling on the GBM workload.
<code>run_localvol_suite.py</code>	LV0–LV10	Runs the local-volatility benchmark suite, including engine comparison, AD-overhead scaling, path-count scaling, time-step scaling, thread scaling, surface-shape sensitivity, timing stability, and local stress testing.
<code>run_profiler_vs_grid.py</code>	<code>sha_cloud_profiler_v4</code>	Runs the final cloud full-grid, old-profiler, and SHA comparison, recording cloud metadata, cost estimates, Pareto-frontier recovery, regret, and promotion decisions.

Primary outputs. Mean runtime, runtime standard deviation, throughput, price estimate, relative price error, memory usage, and environment metadata.

5.10.2 European GBM AD Overhead and Greek Accuracy

This experiment isolates the marginal cost of automatic differentiation on the analytical GBM workload.

Script. `experiments/run_european_ad_analysis.py`

Workload. European GBM.

Engine. JAX/XLA.

Sweep.

$$M \in \{1,000, 5,000, 10,000, 50,000, 100,000\}, \quad \text{AD mode} \in \{\text{none}, \text{forward}, \text{reverse}\}.$$

Purpose. The experiment compares no-AD execution with forward- and reverse-mode AD under identical workload settings. It also checks whether AD-computed Greeks are numerically consistent with the analytical Black–Scholes Delta, Vega, and Rho.

Primary outputs. AD overhead ratio, Greek estimates, Greek errors, runtime, throughput, and memory usage.

5.10.3 European GBM Path-Count Scalability

This experiment measures how each engine behaves as the number of Monte Carlo paths increases on the simple GBM workload.

Script. `experiments/run_european_scalability.py`

Workload. European GBM.

Sweep.

$$M \in \{1,000, 5,000, 10,000, 50,000, 100,000\}.$$

Purpose. The experiment identifies fixed overheads at small path counts and throughput behaviour at larger path counts. It is used as a simple reference before interpreting the more expensive local-volatility workload.

Primary outputs. Runtime, throughput, price error, and memory usage as functions of M .

5.10.4 European GBM Thread Scalability

The GBM thread-scaling experiment measures how engines use local CPU parallelism on a simple workload.

Script. `experiments/run_thread_scalability.py`

Workload. European GBM.

Engines. JAX/XLA and C++/OpenMP, with Rust/Rayon included where supported.

Regimes. Two scaling regimes are evaluated:

- **Strong scaling:** M is fixed while requested thread count varies. Ideal strong scaling means runtime decreases in proportion to the number of threads.
- **Weak scaling:** M increases in proportion to requested thread count. Ideal weak scaling means runtime remains approximately constant as total work and total resources increase together.

Primary outputs. Runtime, throughput, requested thread count, observed thread counts, and scaling efficiency.

5.10.5 Local-Volatility Benchmark Suite

The local-volatility benchmark suite is the main experimental programme in the final evaluation.

Script. `experiments/run_localvol_suite.py`

Workload. European local volatility with default daily discretisation $N = 252$, unless N is explicitly swept.

Engines. NumPy CPU, JAX/XLA, C++/OpenMP, and Rust/Rayon where available. AD modes are evaluated through JAX.

Experiment groups. The suite contains the following experiment groups:

- **LV-0 Smoke test:** checks that all available engines can run the local-volatility workload before expensive measurements are made. This experiment is not reported as a performance result.
- **LV-1 Engine and M -scalability:** compares no-AD throughput across NumPy, JAX, C++, and Rust for $M \in \{10,000, 25,000, 50,000, 100,000, 250,000, 500,000\}$ at $N = 252$.
- **LV-2 JAX AD M -scaling:** compares no AD, forward-mode AD, and reverse-mode AD in JAX for $M \in \{5,000, 10,000, 25,000, 50,000, 100,000, 250,000\}$ at $N = 252$.
- **LV-3 Time-step scaling:** sweeps the number of time steps N at fixed M to separate path-count scaling from discretisation-cost scaling.
- **LV-4 JAX AD time-step scaling:** measures how AD overhead changes as the number of time steps increases.

- **LV-5 Memory pressure:** records peak process memory usage for larger local-volatility runs, with particular attention to JAX AD modes.
- **LV-6 Strong thread scaling:** evaluates whether fixed-size local-volatility workloads speed up when the requested CPU thread count is increased over 1, 2, 4.
- **LV-7 Weak thread scaling:** evaluates whether runtime remains stable when both requested thread count and total path count increase together.
- **LV-8 Surface-shape sensitivity:** compares alternative local-volatility surface presets, such as flat, equity-skew, and strong-smile, where supported by the runner.
- **LV-9 Timing stability:** repeats selected key cells with a larger number of timed runs to estimate measurement variability.
- **LV-10 Stress limit:** tests the largest practical local-volatility configurations that fit within the 4-core, 16 GB RAM Codespace environment.

Purpose. This suite tests whether the performance conclusions from the simple GBM workload still hold for a more realistic MC+AD workload. It provides the primary evidence for programming-language/runtime comparison, AD overhead, path-count scalability, time-step scalability, thread scalability, local resource usage, and measurement stability.

Primary outputs. Mean runtime, runtime standard deviation, throughput, price estimate, AD overhead ratio, memory usage, requested and observed thread counts, and coefficient of variation for selected stability runs.

5.11 Final Cloud and Profiler Experiments

The final experimental extension evaluates cloud cost-performance and the budget-aware profiler. It does not replace the earlier local experiments: those experiments provide the detailed workload, AD, scaling, and resource analysis. The cloud experiment instead asks whether the completed framework can use stored metadata and cheap probes to make cost-aware full-benchmark selection decisions.

5.11.1 Full-Grid Cloud Baseline

The headline full-grid baseline is the `sha_cloud_profiler_v4` experiment. It is recorded at git commit 13789b8 on a Google Cloud `n2-standard-4` instance in `europa-west1-b`, with region `europa-west1`. The full-grid baseline contains 16 supported candidate configurations and three full path counts,

$$M \in \{10,000, 50,000, 100,000\},$$

giving 48 full benchmark cells.

The candidate configurations span the European, Asian, and local-volatility workloads, supported no-AD engines, and JAX AD modes. Unsupported combinations, such as AD for compiled no-AD-only engines, are excluded before the grid is formed. The full grid provides the reference decisions against which profiler quality is judged.

5.11.2 Old Profiler Baseline

The old profiler performs cheap probe runs, ranks candidate configurations, and selects a conservative subset for full benchmarking. In the final experiment it selects 30 of the 48 full-grid cells. Its purpose is to provide a stronger baseline than “no profiling”: SHA must be compared not only with exhaustive search, but also with the earlier probe-selection strategy.

5.11.3 Successive Halving Profiler

The SHA profiler evaluates all candidates at low path counts and promotes a smaller active set to larger probe budgets. The recorded progression uses probe path counts including $M = 1,000$, $M = 5,000$, and $M = 25,000$. After the promotion rounds and conservative safeguards, SHA selects 24 of the 48 full-grid cells for full benchmarking.

The SHA result is evaluated using the same decision-quality criteria as the old profiler: full runs saved, Pareto recovery, runtime regret, cost regret, Spearman probe-to-full rank correlation, and probe path-budget saving. This ensures that the profiler is not credited merely for reducing work; it must also preserve the decisions supported by the full grid.

5.12 Execution Order

The experiments are run in an order that first validates correctness on a simple analytical workload and then moves to the more expensive local-volatility suite and final cloud-profiler evaluation.

1. **European GBM validation:** run the baseline GBM workload and confirm that completed engines produce prices close to the Black–Scholes analytical reference.
2. **European AD validation:** run the JAX forward- and reverse-mode AD experiment and compare Delta, Vega, and Rho against Black–Scholes references.
3. **European path-count and thread scaling:** run the simpler GBM scalability experiments to establish low-cost baseline behaviour.
4. **Local-volatility smoke test:** run LV-0 to confirm that all available engines can execute the multi-step workload.
5. **Local-volatility engine and AD sweeps:** run LV-1 and LV-2 to collect the main local-volatility performance and AD-overhead data.
6. **Local-volatility N -scaling:** run LV-3 and LV-4 to measure how runtime and AD overhead change with the number of time steps.
7. **Local-volatility resource and parallelism experiments:** run LV-5 to LV-10 to evaluate memory usage, thread scaling, surface-shape sensitivity, timing stability, and practical local stress limits.
8. **Cloud full-grid and profiler comparison:** run `sha_cloud_profiler_v4` to compare full-grid benchmarking with the old profiler and SHA under recorded cloud cost metadata.

This order keeps the analytically validated GBM workload as a correctness anchor, while making the local-volatility workload the main performance benchmark for detailed scaling analysis and the cloud-profiler experiment the main evidence for cost-aware selection.

5.13 Database Queries for Analysis

The final analysis uses database queries over completed runs. The exact query predicates may be adjusted to match the experiment labels stored by the runner, but the following views define the main result tables.

5.13.1 European Baseline Throughput

```
SELECT engine,
       mean_runtime_ms,
       std_runtime_ms,
       throughput_paths_per_sec,
       result_value,
       rel_price_error,
       memory_peak_mb
FROM runs
WHERE experiment_type = 'european_baseline'
AND status = 'completed'
```

```
ORDER BY throughput_paths_per_sec DESC;
```

5.13.2 European AD Overhead and Greek Accuracy

```
SELECT M,
       ad_mode,
       mean_runtime_ms,
       ad_overhead_ratio,
       greek_delta,
       analytical_delta,
       rel_delta_error,
       greek_vega,
       analytical_vega,
       rel_vega_error,
       greek_rho,
       analytical_rho,
       rel_rho_error
FROM runs
WHERE experiment_type = 'european_ad_analysis'
      AND status = 'completed'
ORDER BY M, ad_mode;
```

5.13.3 Local-Volatility Engine Scaling

```
SELECT experiment_type,
       engine,
       M,
       N,
       mean_runtime_ms,
       std_runtime_ms,
       throughput_paths_per_sec,
       result_value,
       memory_peak_mb
FROM runs
WHERE experiment_type LIKE 'localvol%'
      AND ad_mode = 'none'
      AND status = 'completed'
ORDER BY experiment_type, M, engine;
```

5.13.4 Local-Volatility AD Overhead

```
SELECT experiment_type,
       M,
       N,
       ad_mode,
       mean_runtime_ms,
       std_runtime_ms,
       ad_overhead_ratio,
       throughput_paths_per_sec,
       memory_peak_mb,
       result_value
FROM runs
WHERE experiment_type LIKE 'localvol%'
      AND engine = 'jax'
      AND status = 'completed'
ORDER BY experiment_type, N, M, ad_mode;
```

5.13.5 Thread Scaling

```
SELECT engine,
       experiment_type,
       requested_threads,
       observed_threads_max,
       M,
       N,
       mean_runtime_ms,
       throughput_paths_per_sec
FROM runs
WHERE experiment_type LIKE '%thread%'
      AND status = 'completed'
ORDER BY experiment_type, engine, M, N, requested_threads;
```

5.14 Reproducibility

Each completed run stores the full workload configuration and environment metadata needed to reproduce or audit the measurement. The database record includes

- the serialised workload configuration;
- a configuration hash where available;
- engine, language, backend, and AD mode;
- path count, time-step count, option parameters, and seed;
- local-volatility surface parameters or preset;
- timing statistics;
- price and Greek outputs where available;
- analytical reference values for GBM runs;
- numerical error metrics where a reference is available;
- memory usage;
- requested and observed thread counts where available;
- CPU, operating-system, Python, NumPy, and JAX metadata.

This metadata ensures that every reported number can be traced back to a specific workload, software stack, and hardware environment. The experiment database is append-only during collection: failed, skipped, and completed runs are all preserved so that the final analysis does not depend on unrecorded manual filtering.

5.15 Scope of Excluded Extensions

The original project proposal included cloud resource selection, GPU acceleration, distributed execution, mixed precision, BLAS backend sweeps, and vendor-claim validation through emerging languages such as Mojo. Cloud resource selection is partially addressed through the final cost-performance and profiler experiments. GPU acceleration, distributed execution, mixed precision, broader BLAS-environment sweeps, and Mojo vendor-claim validation remain future extensions.

The final results therefore combine local CPU benchmarking across NumPy, JAX/XLA, C++/OpenMP, and Rust/Rayon with a targeted cloud-profiler study. The local campaign provides detailed analysis of local-volatility performance, AD overhead, path-count scaling, time-step scaling, thread scaling, and measurement stability. The cloud campaign provides measured cost-performance, Pareto-frontier, and profiler-selection evidence for the completed framework.

5.16 Summary

The experimental setup defines a controlled methodology for MC+AD workloads, combining a 4-core, 16 GB RAM Codespace campaign with final Google Cloud profiler experiments. The European GBM workload provides analytical validation and a simple baseline. The parametric local-volatility workload is the main detailed performance benchmark, adding multi-step simulation, nonlinear model structure, and higher AD complexity. The Asian workload adds a path-dependent case to the cloud-profiler grid.

Together, the experiments evaluate runtime performance, scalability, automatic differentiation overhead, numerical consistency, thread behaviour, local resource usage, cloud cost-performance, and budget-aware profiler quality across NumPy, JAX/XLA, C++/OpenMP, and Rust/Rayon where supported. The next chapter reports the measurements obtained from these experiments and discusses how far they answer the project research questions.

Chapter 6

Results and Discussion

6.1 Overview

This chapter reports and interprets the benchmark results obtained from the experimental setup in Chapter 5. The European GBM workload is used as a validation and calibration workload: it verifies that the framework produces credible Monte Carlo prices and automatic differentiation outputs against a known Black–Scholes reference. The main performance analysis is then carried out on the parametric local-volatility workload, which is more representative of the target MC+AD setting because it includes time stepping, a nonlinear volatility surface, and a larger differentiation graph.

The results combine two layers of evidence. The first layer is a detailed local CPU study on the Codespace environment, comparing NumPy, JAX/XLA, C++/OpenMP, and Rust/Rayon where available. This provides the main evidence for validation, workload scaling, AD overhead, memory pressure, and thread behaviour. The second layer is the final cloud-profiler study, which adds Google Cloud cost-performance measurements, Pareto-frontier analysis, and old-profiler versus SHA selection quality. GPU acceleration, distributed execution, broad BLAS-environment sweeps, mixed precision, and Mojo vendor-claim validation are not reported as completed experiments in this chapter.

6.2 European GBM Validation

The European GBM workload is not the main performance workload in this chapter. Its role is to provide a simple analytical reference before interpreting the more expensive local-volatility experiments. For the standard parameter set $S_0 = K = 100$, $r = 0.05$, $\sigma = 0.20$, and $T = 1$, the Black–Scholes analytical call price is 10.4506, with Delta 0.6368, Vega 37.524, and Rho 53.232.

The baseline GBM results show that all completed engines produce prices close to the analytical Black–Scholes value. The C++ engine is the fastest on this small single-step workload, achieving 101.6 million paths/s, while NumPy reaches 45.5 million paths/s and JAX reaches 17.9 million paths/s. This result is useful as a sanity check, but it is not treated as the main performance story because the workload is too small and too simple to represent the multi-step MC+AD case.

The JAX AD validation also produced accurate Greeks at larger path counts. At $M = 100,000$, forward- and reverse-mode AD gave the same reported Greek estimates to the displayed precision. Delta error was 0.000428 (0.07%), Vega error was 0.259 (0.69%), and Rho error was 0.080 (0.15%). This confirms that the JAX AD implementation is numerically credible before it is used for the local-volatility AD experiments.

6.3 Local-Volatility Smoke Test

LV-0 is a smoke test and is not used as a main performance result. It checks that each available engine can execute the local-volatility workload and write results to the shared database. The test uses only one timed run at $M = 1,000$, so the timings are not statistically meaningful.

The smoke test confirms that the local-volatility pipeline runs across all four engines. The Rust price is visibly different from the other engines at this very small path count, but this is not

Table 6.1: Result sections and their corresponding evidence.

Section	Experiment source	Main evidence
European GBM validation	European baseline and AD analysis	Black-Scholes price and Greek agreement used to validate the framework.
Local-volatility engine scaling	LV-1	No-AD runtime scaling across NumPy, JAX, C++, and Rust as path count increases.
Local-volatility AD overhead	LV-2 and LV-4	Forward- and reverse-mode JAX AD overhead as a function of path count M and time-step count N .
Time-step scalability	LV-3	Runtime scaling with the number of local-volatility time steps.
Memory and stress limits	LV-5 and LV-10	Resource limits for JAX AD, including observed out-of-memory failures.
Thread scalability	LV-6 and LV-7	Strong and weak scaling for C++/OpenMP, Rust/Rayon, and JAX/XLA.
Surface-shape sensitivity	LV-8	Runtime effect of changing the local-volatility surface preset.
Timing stability	LV-9	Coefficient of variation over 20 repeated runs for key benchmark cells.
Final cloud cost-performance	<code>sha_cloud_profiler_v4</code> and generated cloud artefacts	Runtime, estimated cost per run, and best no-AD configurations for priced cloud instances.
Profiler selection quality	Full-grid, old-profiler, and SHA comparison	Full runs saved, Pareto recovery, runtime regret, cost regret, rank correlation, and SHA promotion behaviour.

Table 6.2: European GBM baseline validation at $M = 10,000$.

Engine	Price	Mean ms	Std ms	Paths/s	Rel. price error
NumPy/CPU	10.3452	0.220	0.016	45.5M	1.01%
JAX/XLA	10.2098	0.559	0.051	17.9M	2.30%
C++/OpenMP	10.4658	0.098	0.009	101.6M	0.15%

Table 6.3: LV-0: smoke test ($M = 1,000$, $N = 252$, one timed run).

Engine	AD mode	Latency ms	Price	Delta / Vega
<code>cpu</code>	none	16.000	10.61771	—
<code>cpp</code>	none	7.427	10.79929	—
<code>rust</code>	none	5.095	12.19688	—
<code>jax</code>	none	10.205	10.33823	—
<code>jax</code>	forward	45.499	10.33823	$\delta = 0.6294$, $\nu = 37.49$
<code>jax</code>	reverse	23.859	10.33823	$\delta = 0.6294$, $\nu = 37.49$

interpreted as a correctness failure because $M = 1,000$ is too small for a reliable local-volatility cross-engine comparison and different engines may use different random number generators. The larger M -sweeps below are used for performance interpretation.

6.4 Local-Volatility Engine Scalability

LV-1 is the main no-AD engine comparison on the local-volatility workload. The workload uses $N = 252$ time steps and sweeps path count from 10,000 to 500,000.

Table 6.4: LV-1: engine scalability on local volatility ($N = 252$, no AD).

Engine	$M = 10k$ ms	$M = 25k$ ms	$M = 50k$ ms	$M = 100k$ ms	$M = 250k$ ms	$M = 500k$ ms
cpu	100.7 ± 3.5	255.8 ± 7.6	501.5 ± 10.4	995.4 ± 24.7	2448.3 ± 25.7	5022.4 ± 52.0
c++	69.8 ± 12.9	158.8 ± 19.2	338.6 ± 45.4	633.4 ± 41.8	1621.0 ± 61.9	3062.6 ± 82.1
rust	57.3 ± 9.4	122.1 ± 11.9	239.3 ± 14.1	497.1 ± 32.2	1227.7 ± 55.0	2439.6 ± 57.3
jax	93.4 ± 12.6	199.6 ± 41.4	339.3 ± 47.8	629.9 ± 24.2	1558.3 ± 41.0	3103.7 ± 45.0

The local-volatility workload shows approximately linear runtime growth with path count for all engines. This is expected because the workload cost is roughly proportional to $M \times N$, and N is fixed at 252 in this experiment.

Rust/Rayon is the fastest no-AD engine across the full path-count sweep. At $M = 500,000$, Rust completes the workload in 2439.6 ms, compared with 3062.6 ms for C++, 3103.7 ms for JAX, and 5022.4 ms for NumPy. This corresponds to approximately 0.205 million paths/s for Rust, 0.163 million paths/s for C++, 0.161 million paths/s for JAX, and 0.100 million paths/s for NumPy. Thus, on this local-volatility workload, Rust is approximately $2.1\times$ faster than NumPy and about $1.25\times$ faster than JAX or C++ at the largest tested path count.

A notable difference from the simple European GBM workload is that JAX becomes competitive with C++ on the more expensive multi-step workload. This suggests that the fixed overheads of XLA execution are less important once each benchmark cell contains enough path-level and time-step work.

6.5 Local-Volatility AD Scaling with Path Count

LV-2 measures the cost of JAX forward- and reverse-mode AD on the local-volatility workload as the number of paths increases. All runs use $N = 252$.

Table 6.5: LV-2: JAX AD path-count scaling on local volatility ($N = 252$).

M	None ms	Forward ms	Reverse ms	Forward OH	Reverse OH
5k	42.4 ± 0.2	210.4 ± 4.6	90.3 ± 4.9	$4.94\times$	$2.13\times$
10k	98.8 ± 19.8	437.0 ± 7.6	179.4 ± 5.2	$5.02\times$	$2.08\times$
25k	166.1 ± 20.7	985.7 ± 51.2	373.8 ± 14.3	$6.00\times$	$2.35\times$
50k	340.9 ± 40.9	1511.3 ± 31.9	654.6 ± 20.5	$4.87\times$	$2.10\times$
100k	644.1 ± 25.8	3121.3 ± 106.0	1312.0 ± 35.2	$4.86\times$	$2.06\times$
250k	1558.2 ± 24.0	6252.4 ± 109.7	3627.0 ± 82.6	$4.08\times$	$2.34\times$

The main AD result is clear: reverse-mode AD is substantially cheaper than forward-mode AD for this scalar-output local-volatility pricing workload. Across the tested path counts, forward-mode overhead is roughly $4.1\text{--}6.0\times$, while reverse-mode overhead is roughly $2.1\text{--}2.4\times$. This is consistent with the expected AD trade-off: the pricing function has many input parameters but produces a scalar price, making reverse mode a more suitable differentiation strategy.

The overhead does not vanish at large M . At $M = 250,000$, reverse mode still costs $2.34\times$ the no-AD runtime, while forward mode costs $4.08\times$. This shows that AD remains a first-order performance concern for MC workloads even when the underlying simulation is large enough to amortise JAX/XLA dispatch overheads.

6.6 Local-Volatility Scaling with Time Steps

LV-3 separates path-count scaling from time-discretisation scaling by fixing $M = 50,000$ and varying the number of local-volatility time steps.

Table 6.6: LV-3: time-step scaling on local volatility ($M = 50,000$, no AD).

Engine	$N = 12$	$N = 52$	$N = 126$	$N = 252$	$N = 504$
cpp	17.6 ± 5.9	66.7 ± 8.3	170.2 ± 25.7	328.0 ± 49.3	633.2 ± 32.2
rust	12.8 ± 3.1	49.9 ± 3.9	132.3 ± 7.5	256.1 ± 18.6	519.4 ± 63.0
jax	20.5 ± 3.8	61.7 ± 4.8	171.4 ± 27.3	324.0 ± 40.8	648.7 ± 39.5

Runtime increases approximately linearly with N , which confirms that the local-volatility workload is dominated by repeated time-step updates rather than a fixed one-off overhead. For example, Rust increases from 256.1 ms at $N = 252$ to 519.4 ms at $N = 504$, close to a $2\times$ increase when the number of time steps doubles. JAX and C++ show the same general pattern.

Rust remains the fastest engine in this experiment, followed by C++ and JAX. The difference between C++ and JAX is small at $N = 252$, but both are slower than Rust. This reinforces the LV-1 result that the Rust/Rayon implementation is the strongest no-AD local-volatility engine in the current local environment.

6.7 Local-Volatility AD Scaling with Time Steps

LV-4 measures how JAX AD overhead changes with the number of time steps. The path count is fixed at $M = 25,000$.

Table 6.7: LV-4: JAX AD time-step scaling on local volatility ($M = 25,000$).

N	None ms	Forward ms	Reverse ms	Forward OH	Reverse OH
12	7.5 ± 0.7	50.1 ± 3.5	18.6 ± 0.8	$7.14\times$	$2.47\times$
52	28.7 ± 0.9	203.3 ± 3.8	73.9 ± 2.2	$5.11\times$	$2.51\times$
126	72.6 ± 2.0	504.7 ± 27.0	186.9 ± 9.8	$6.81\times$	$2.62\times$
252	156.7 ± 5.3	974.6 ± 15.8	372.4 ± 14.4	$6.62\times$	$2.52\times$
504	329.5 ± 27.7	1901.5 ± 13.5	738.4 ± 14.7	$6.32\times$	$2.33\times$

The AD overhead pattern is stable across simulation length. Reverse mode remains close to $2.3\text{--}2.6\times$ the no-AD runtime, while forward mode remains much more expensive, usually above $5\times$. Increasing the number of time steps therefore increases absolute runtime for all modes, but does not change the conclusion that reverse mode is the better AD strategy for this scalar-output workload.

The $N = 504$ case is particularly important because it doubles the default daily time-step count. Even there, reverse mode completes in 738.4 ms, compared with 1901.5 ms for forward mode. This gives a strong practical argument for reverse-mode AD when many sensitivities are required from a single price output.

6.8 Memory Pressure and Practical Resource Limits

Memory behaviour is analysed separately because runtime alone does not show whether a method remains usable at larger M and N . This matters especially for reverse-mode AD, where intermediate values from the scan may need to be retained or reconstructed during the backward pass.

The most important memory result is not the small process-level RSS deltas, which are noisy under JAX/XLA, but the observed out-of-memory boundary. JAX reverse mode succeeds at $M = 100k$, $N = 504$, but fails at $M = 250k$, $N = 504$. It also succeeds at $M = 500k$, $N = 252$ in the stress test below, but fails at $M = 1M$, $N = 252$. These failures provide direct evidence that reverse-mode AD has a practical memory envelope on the 4-core, 16 GB Codespace environment.

Forward mode is slower than reverse mode, but it completes all LV-5 cells, including $M = 250k$, $N = 504$. This gives a useful trade-off: reverse mode is substantially faster for moderate problem sizes, but forward mode may be more robust in some large memory-stressed settings.

Table 6.8: LV-5: JAX memory-pressure experiment.

N	AD mode	$M = 25k$ ms	$M = 50k$ ms	$M = 100k$ ms	$M = 250k$ ms
252	none	152.4 ± 5.7	315.2 ± 23.2	650.8 ± 38.4	1527.9 ± 20.5
	reverse	380.1 ± 21.1	649.4 ± 22.2	1332.9 ± 64.3	3573.5 ± 51.4
	forward	961.8 ± 16.9	1515.7 ± 38.3	3103.6 ± 62.7	6278.7 ± 161.8
504	none	334.5 ± 61.9	647.6 ± 32.6	1239.6 ± 24.9	3102.3 ± 58.6
	reverse	754.8 ± 12.3	1310.9 ± 26.0	2595.1 ± 78.6	OOM
	forward	1979.6 ± 100.8	3006.0 ± 49.3	6101.9 ± 133.7	12392.9 ± 121.3

OOM: the JAX reverse-mode cell at $M = 250k$, $N = 504$ was killed by the kernel out-of-memory mechanism.

6.9 Thread Scalability on Local Volatility

LV-6 and LV-7 evaluate CPU thread scaling on the local-volatility workload. The experiments use requested thread counts $T = 1, 2, 4$. C++ and Rust use explicit OpenMP/Rayon parallelism, while JAX uses XLA’s own CPU execution runtime and is not expected to respond directly to OMP_NUM_THREADS.

6.9.1 Strong Scaling

Table 6.9: LV-6: strong thread scaling ($M = 100,000$, $N = 252$).

Engine	$T = 1$ ms	$T = 2$ ms	$T = 4$ ms	S_2	S_4	Efficiency ₄
cpp	1944.4 ± 37.3	973.9 ± 5.9	629.9 ± 31.7	$2.00\times$	$3.09\times$	77.2%
rust	1787.3 ± 2.7	900.0 ± 4.1	488.1 ± 34.6	$1.99\times$	$3.66\times$	91.5%
jax	642.4 ± 20.7	637.5 ± 32.3	642.7 ± 19.4	$1.01\times$	$1.00\times$	25.0%

$S_k = T_1/T_k$. JAX uses XLA’s own CPU parallelism and does not show meaningful sensitivity to the requested OpenMP thread count.

C++ and Rust both scale strongly with requested thread count. Rust improves from 1787.3 ms at one thread to 488.1 ms at four threads, giving a $3.66\times$ speedup and 91.5% four-thread efficiency. C++ improves from 1944.4 ms to 629.9 ms, giving a $3.09\times$ speedup and 77.2% efficiency. This shows that the local-volatility workload exposes substantial path-level parallelism to compiled CPU engines.

JAX does not scale with the requested thread count in this experiment. Its runtime remains close to 640 ms for $T = 1, 2, 4$. This does not mean JAX is single-threaded; rather, it suggests that XLA manages CPU parallelism internally and is not controlled by the same OpenMP/Rayon knobs used by C++ and Rust.

6.9.2 Weak Scaling

Table 6.10: LV-7: weak thread scaling (base $M = 50,000$ paths/thread, $N = 252$).

Engine	$T = 1$, $M = 50k$ ms	$T = 2$, $M = 100k$ ms	$T = 4$, $M = 200k$ ms	TP speedup $1 \rightarrow 4$
cpp	961.8 ± 7.6	975.2 ± 13.8	1238.7 ± 29.6	$3.11\times$
rust	893.6 ± 2.1	913.4 ± 33.7	969.9 ± 34.2	$3.68\times$
jax	316.2 ± 16.1	675.0 ± 95.8	1252.0 ± 32.1	$1.01\times$

Throughput speedup is computed as $(4M_{\text{base}}/T_4)/(M_{\text{base}}/T_1)$. Ideal weak scaling would give $4\times$ throughput at approximately constant latency.

Weak scaling again favours the compiled engines. Rust achieves a $3.68\times$ throughput speedup from one to four requested threads, while C++ achieves $3.11\times$. Latency is not perfectly constant, especially for C++ at $T = 4$, but both compiled engines make effective use of additional requested threads.

JAX shows almost no weak-scaling throughput gain under the requested thread sweep. Its latency increases from 316.2 ms at $M = 50k$ to 1252.0 ms at $M = 200k$, giving only $1.01\times$

throughput speedup. This supports the interpretation that JAX/XLA’s CPU execution strategy is not controlled by the same explicit thread-count mechanism as the compiled OpenMP/Rayon engines.

6.10 Surface-Shape Sensitivity

LV-8 tests whether changing the local-volatility surface shape materially changes runtime. The experiment compares flat, equity-skew, and strong-smile presets at $M = 50,000$, $N = 252$.

Table 6.11: LV-8: surface-shape sensitivity ($M = 50,000$, $N = 252$).

Engine	AD	Flat ms	Equity-skew ms	Strong-smile ms	Reverse OH
jax	none	314.9 ± 26.8	318.1 ± 24.6	325.5 ± 14.8	—
	reverse	669.9 ± 47.3	663.4 ± 27.4	682.4 ± 77.1	2.09–2.15×
cpp	none	347.8 ± 64.5	327.9 ± 14.4	341.2 ± 75.3	—

JAX prices were 10.38974, 10.39082, and 10.40619 for the flat, equity-skew, and strong-smile presets respectively. C++ prices were 10.44431, 10.44826, and 10.45843.

Surface shape has little effect on runtime. For JAX no-AD execution, the three presets all complete between 314.9 ms and 325.5 ms. For JAX reverse mode, overhead remains in a narrow range of 2.09–2.15×. This suggests that the cost of the local-volatility workload is dominated by the number of paths, number of time steps, random number generation, and scan structure, rather than by the particular parameter values used in the polynomial volatility surface.

This experiment is still useful because it demonstrates that the framework can vary model structure while keeping the same execution harness. That is important for extensibility: new model presets can be added without changing the benchmarking methodology.

6.11 Timing Stability

LV-9 repeats selected key cells for 20 timed runs and reports the coefficient of variation (CV). This checks whether the benchmark results are stable enough to support comparisons.

Table 6.12: LV-9: timing stability ($M = 100,000$, $N = 252$, 20 timed runs).

Engine	AD	Mean ms	Std ms	Min ms	Max ms	CV
jax	none	629.5	29.9	596.4	722.9	4.75%
jax	reverse	1275.3	41.2	1227.5	1390.2	3.23%
cpp	none	632.7	43.4	594.0	739.4	6.86%
rust	none	486.3	22.2	459.9	529.9	4.56%

$$CV = \sigma/\mu \times 100\%.$$

The timing stability experiment shows acceptable variability for a shared Codespace environment. CV values range from 3.23% for JAX reverse mode to 6.86% for C++. This is stable enough to support the main engine and AD comparisons, especially because the observed performance differences between engines and AD modes are much larger than the measured run-to-run variation.

Rust is both the fastest no-AD engine in this stability subset and one of the more stable engines, with a CV of 4.56%. JAX reverse mode is the most consistent cell, with a CV of 3.23%, likely because the compiled execution path is stable after warmup.

6.12 Stress Limit

LV-10 probes the largest local-volatility workloads attempted on the Codespace environment. It compares the fastest compiled no-AD engines against JAX reverse-mode AD at $N = 252$.

The stress test reinforces two conclusions. First, Rust remains faster than C++ at large local-volatility path counts: at $M = 1M$, Rust completes in 5028.3 ms, while C++ takes 6363.0 ms.

Table 6.13: LV-10: stress limit ($N = 252$, large M).

Engine	AD	$M = 500k$ ms	Mem MB	$M = 1M$ ms
cpp	none	3099.3 ± 41.9	—	6363.0 ± 202.2
rust	none	2422.2 ± 36.7	—	5028.3 ± 36.9
jax	reverse	6878.3 ± 103.9	527.9	OOM

JAX reverse mode at $M = 1M$, $N = 252$ was killed by the kernel out-of-memory mechanism.

Second, JAX reverse-mode AD has a practical memory limit in the current environment. It completes $M = 500k$, $N = 252$, but fails at $M = 1M$, $N = 252$.

Together with LV-5, the OOM cases locate the practical JAX reverse-mode boundary between successful $M = 500k$, $N = 252$ and failed $M = 1M$, $N = 252$, and between successful $M = 100k$, $N = 504$ and failed $M = 250k$, $N = 504$. This is one of the most important resource-utilisation findings in the chapter: reverse-mode AD is faster than forward mode for moderate local-volatility workloads, but memory becomes the limiting factor at larger path-count/time-step products.

6.13 Final Cloud Cost-Performance and Profiler Results

The local experiments above explain the detailed behaviour of the framework on controlled CPU workloads. The final cloud experiment extends that evidence by asking whether the framework can make cost-aware benchmark-selection decisions from stored cloud metadata and cheap probes. The headline experiment is `sha_cloud_profiler_v4`, run at git commit `13789b8` on Google Cloud `n2-standard-4` in `eu-west1-b`. The full-grid baseline contains 16 candidate configurations evaluated at three full path counts, for 48 full benchmark cells.

6.13.1 Correctness and AD Overhead in the Final Cloud Run

The European GBM workload is again used as the analytical validation point. Table 6.14 shows the stored JAX AD validation at $M = 100,000$. Forward and reverse mode produce the same reported price and Greeks in this experiment. The price error is 0.357%, and the absolute errors for Delta, Vega, and Rho are small relative to the scale of the reported Greeks.

Table 6.14: European GBM AD validation against Black–Scholes analytical values at $M = 100,000$.

AD mode	Price err.	Delta err.	Vega err.	Rho err.
forward	0.357%	0.000428	0.259	0.080
reverse	0.357%	0.000428	0.259	0.080

Table 6.15 reports JAX AD overhead at $M = 100,000$ on `n2-standard-4`. The final cloud results broaden the earlier local-volatility-only AD discussion by including European, Asian, and local-volatility workloads in the same cloud comparison.

Table 6.15: JAX automatic differentiation overhead at $M = 100,000$ on `n2-standard-4`.

Workload	AD mode	Runtime ms	Overhead
Asian	reverse	1148.4	$1.81\times$
European	forward	4.783	$1.96\times$
European	reverse	5.208	$2.10\times$
Asian	forward	1552.1	$2.45\times$
Local volatility	reverse	2169.7	$2.55\times$
Local volatility	forward	3768.5	$4.42\times$

The measured overheads range from $1.81\times$ for Asian reverse mode to $4.42\times$ for local-volatility forward mode. The ordering is workload dependent: reverse mode is cheaper than forward mode for the Asian and local-volatility measurements, while the European forward and reverse overheads

are closer. This reinforces the earlier conclusion that AD mode should be measured under the actual workload rather than chosen by a fixed rule of thumb.

6.13.2 Cloud Cost-Performance

Table 6.16 summarises the best no-AD cost-performance rows for priced cloud instances at $M = 100,000$. It reports both runtime and estimated cost per run, making explicit that a faster machine can also be cheaper per run if the runtime improvement outweighs the hourly price difference.

Table 6.16: Best no-AD cost-performance per priced cloud instance at $M = 100,000$.

Workload	Instance	Best engine	Runtime ms	Cost/run USD	M paths/s
Asian	n2-standard-4	jax/none	634.6	3.42e-05	0.158
Asian	t2d-standard-4	jax/none	343.9	1.62e-05	0.291
European	n2-standard-4	rust/none	0.539	2.91e-08	185.495
European	t2d-standard-4	rust/none	0.356	1.68e-08	281.263
Local volatility	n2-standard-4	rust/none	488.9	2.64e-05	0.205
Local volatility	t2d-standard-4	rust/none	443.6	2.10e-05	0.225

Figure 6.1 visualises the same relationship. In the stored priced comparison, **t2d-standard-4** is faster and lower cost per run than **n2-standard-4** for the listed best no-AD workloads. This is an empirical result for the measured workloads and pricing assumptions, not a claim about all machine types or future cloud prices.

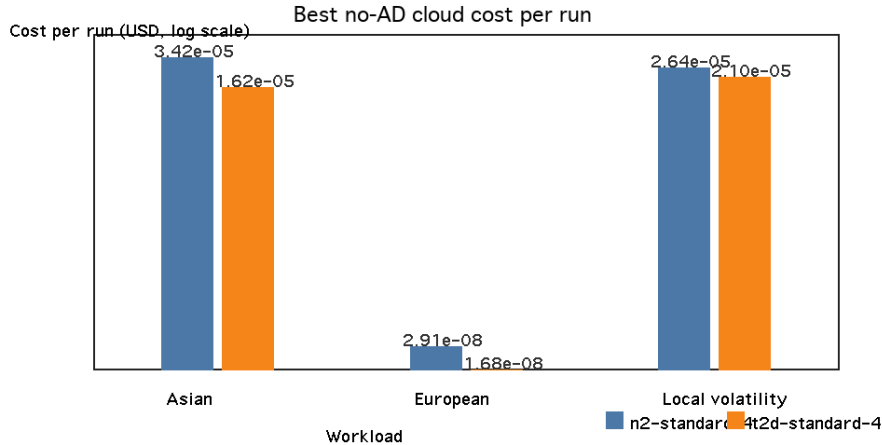


Figure 6.1: Runtime and estimated cost per run for best no-AD cloud configurations at $M = 100,000$.

6.13.3 Pareto Frontier

Figure 6.2 plots runtime against estimated cost for the final **n2-standard-4** full-grid comparison. Because the plotted configurations are on the same instance type, runtime and cost are closely linked. The value of the Pareto analysis is therefore not that it reveals a complex two-dimensional trade-off on one machine, but that it gives a precise target for profiler recovery: a profiler should retain the configurations that the full grid identifies as non-dominated under the measured metrics.

6.13.4 Old Profiler versus SHA

Table 6.17 is the main final cloud result. The full grid contains 48 full benchmark runs. The old profiler selects 30 runs, saving 37.5%. SHA selects 24 runs, saving 50.0%. Both profilers recover 100.0% of the full-grid Pareto frontier and incur 0.0% runtime and cost regret relative to the

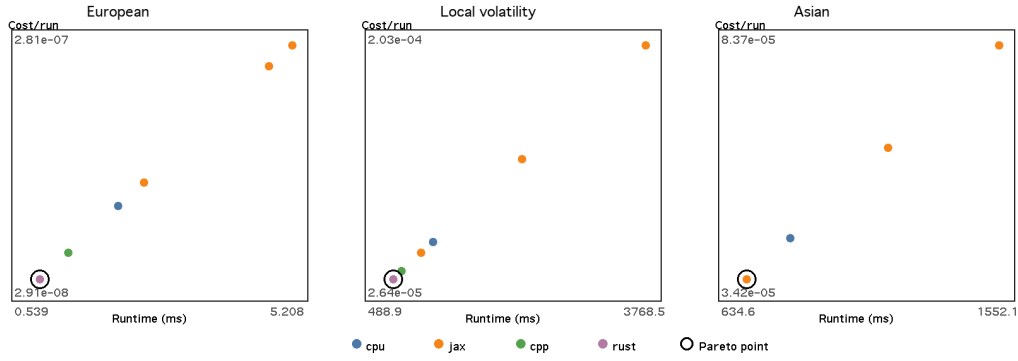


Figure 6.2: Runtime versus estimated cost for the final **n2-standard-4** grid. Pareto-optimal points are highlighted.

full-grid best. SHA also achieves a stored final probe-to-full Spearman rank correlation of 1.000, compared with 0.935 for the old profiler evidence.

Table 6.17: Full-grid, old-profiler and SHA comparison for the final **n2-standard-4** cloud experiment.

Metric	Full grid	Old profiler	SHA
Candidate configurations	16	16	16
Full benchmark runs	48	30	24
Full runs saved	0	18	24
Percentage saved	0.0%	37.5%	50.0%
Pareto recovery	baseline	100.0%	100.0%
Runtime regret	baseline	+0.0%	+0.0%
Cost regret	baseline	+0.0%	+0.0%
Spearman ρ probe-full	—	0.935	1.000
Probe path-budget saving	—	—	95.5%
Mean scaling-law error	—	—	9.7%

This result directly addresses the cloud resource-selection goal of the project. In the measured grid, SHA halves the number of full-scale cloud benchmark cells while preserving the full-grid decision quality on the reported metrics. The practical significance is that the expensive part of the experiment is not merely shifted elsewhere: the SHA probe path budget is 95.5% lower than evaluating the grid at full scale, and the mean scaling-law extrapolation error is 9.7%.

6.13.5 SHA Progression

Figure 6.3 shows the SHA promotion sequence recorded for the final experiment. All 16 configurations are evaluated at the cheapest probe level, then the active set is reduced as the path budget increases. The final selected set contains the configurations promoted for full benchmarking after the profiler’s conservative safeguards are applied.

The progression illustrates why SHA is useful in this setting. Most configurations receive only cheap measurements, while promising configurations receive more budget. In the final experiment this is enough to identify the same best runtime configuration as the full grid: **rust/european/none** at $M = 100,000$, with runtime 0.54 ms.

6.14 Cross-Question Discussion

6.14.1 Software Stack and Engine Performance

The local-volatility results show that engine choice has a substantial effect on throughput. Rust/Rayon is the fastest no-AD engine across the main M -sweep and the large stress test. At $M = 500k$, $N = 252$, Rust is about $2.1\times$ faster than NumPy and about $1.25\times$ faster than JAX

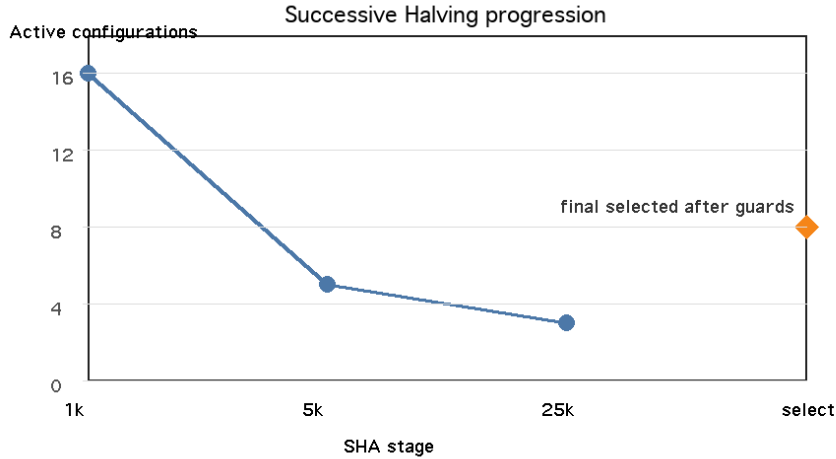


Figure 6.3: SHA active configuration count as probe path budget increases.

or C++. JAX and C++ are very similar at larger local-volatility sizes, while NumPy remains consistently slower.

The result also shows why the European GBM workload should not be treated as the main benchmark. On the small GBM baseline, C++ was clearly dominant. On local volatility, Rust becomes fastest and JAX becomes competitive with C++. This demonstrates that engine rankings depend strongly on workload structure.

6.14.2 Programming-Language and AD Trade-offs

The compiled CPU engines provide the strongest no-AD local-volatility performance. Rust/Rayon performs best overall in the local-volatility experiments, while C++/OpenMP remains competitive and scales well with threads. JAX/XLA is not the fastest no-AD engine, but it provides the main AD capability and becomes close to C++ on larger multi-step workloads.

The AD results show a clear trade-off. Reverse-mode AD is consistently much faster than forward-mode AD for the scalar-output local-volatility pricing function, with reverse-mode overhead usually around 2.1–2.6 $\times$ and forward-mode overhead around 4–7 $\times$. However, reverse mode also reaches memory limits at larger M, N combinations. Therefore, the best AD choice depends on whether the bottleneck is runtime or memory.

6.14.3 Parallelism and Bottlenecks

The thread-scaling results show that C++ and Rust expose path-level parallelism effectively to OpenMP and Rayon. Rust achieves 3.66 $\times$ strong-scaling speedup at four requested threads and 3.68 $\times$ weak-scaling throughput speedup. C++ also scales well, though less strongly, with 3.09 $\times$ strong-scaling speedup and 3.11 $\times$ weak-scaling throughput speedup.

JAX behaves differently. It is competitive in absolute runtime for no-AD local-volatility execution, but it does not respond to requested thread-count changes. This indicates that XLA’s managed CPU runtime should be interpreted separately from explicit OpenMP/Rayon thread scaling. For this project, the main bottlenecks are therefore different by engine: compiled engines are limited by parallel efficiency and memory bandwidth at higher thread counts, while JAX is limited by its managed execution model and, for reverse-mode AD, by memory pressure.

6.14.4 AI/ML Compiler Techniques

The JAX/XLA results provide partial evidence for the relevance of AI/ML compiler techniques to Monte Carlo simulation. JAX is relatively weak on the small European GBM baseline, but becomes competitive with C++ on the multi-step local-volatility workload. This suggests that XLA-style compiled execution becomes more attractive when the workload has enough repeated structure to amortise dispatch and compilation overheads.

However, the results do not test mixed precision, kernel auto-tuning, GPU execution, or hardware-aware scheduling beyond JAX/XLA’s CPU backend. Therefore, the AI/ML-technique conclusion should be limited to compiled XLA execution and automatic differentiation support, not to the full range of AI-system optimisation techniques.

6.14.5 Scope Against the Original Proposal

The completed experiments strongly address local MC+AD benchmarking, programming-language comparison, AD overhead, memory/resource limits, thread scalability, and a targeted form of cloud resource selection. The final cloud-profiler study adds measured cost-performance, Pareto-frontier, and profiler-selection evidence. GPU acceleration, distributed execution, broad BLAS-environment comparison, and Mojo vendor-claim validation remain valid extensions of the framework but are outside the completed result set.

This scope is scientifically useful. The project demonstrates a reproducible benchmarking framework that can run multiple engines under matched workload definitions, capture repeated measurements, evaluate AD overheads, reveal practical resource limits, and attach cloud cost metadata to final selection decisions. The local-volatility suite also shows that more realistic MC+AD workloads can change the conclusions drawn from simple one-step baselines.

6.15 Limitations and Threats to Validity

6.15.1 Hardware and Cloud Scope

The detailed scaling and memory experiments were collected on a single Codespace environment. This makes the comparisons controlled and reproducible under the recorded conditions, but it prevents broad claims about AMD versus Intel, workstation versus cloud, or CPU versus GPU performance. The final cloud experiments add targeted Google Cloud evidence, but those results are still tied to specific instance types, regions, prices, software versions, and run dates. They should therefore be interpreted as measured cost-performance evidence for the reported configurations, not as universal cloud hardware rankings.

6.15.2 Local-Volatility Correctness

The European GBM workload has a closed-form Black–Scholes reference, but the chosen parametric local-volatility workload does not. Local-volatility correctness is therefore assessed through cross-engine consistency and convergence with larger M , rather than by exact analytical error. Price differences between engines may also reflect different random-number generators or path-generation implementations.

6.15.3 Memory Measurement

Process-level memory measurements are coarse, especially for JAX/XLA, which may cache allocations or materialise intermediates inside compiled execution. For this reason, small RSS deltas are not over-interpreted. The OOM events are treated as stronger evidence than individual low memory-delta readings.

6.15.4 Thread Control

Thread-count control is more direct for C++/OpenMP and Rust/Rayon than for JAX/XLA. JAX uses its own CPU execution runtime, so requested OpenMP-style thread counts do not translate into clean JAX scaling experiments. This is why JAX thread results are interpreted as runtime behaviour rather than as a controlled thread-count benchmark.

6.15.5 Excluded Proposal Extensions

The original project proposal included cloud resource selection, GPU acceleration, distributed execution, mixed precision, BLAS backend sweeps, and Mojo vendor-claim validation. Cloud resource selection is partially completed through the final cost-performance and profiler results.

GPU acceleration, distributed execution, mixed precision, wider BLAS sweeps, and Mojo vendor-claim validation are future extensions that the framework is designed to support, but they are not part of the completed experimental evidence.

6.16 Summary

The main findings are as follows.

- **European GBM validation:** the simple GBM workload verifies that the framework produces credible prices and AD Greeks against Black–Scholes references. It is useful as a validation workload, but not representative enough to be the main benchmark.
- **Local-volatility engine performance:** Rust/Rayon is the fastest no-AD local-volatility engine across the main M -sweep and the large stress test. JAX/XLA and C++/OpenMP are close at larger local-volatility sizes, while NumPy is consistently slower.
- **AD overhead:** reverse-mode AD is substantially cheaper than forward-mode AD for the scalar-output local-volatility pricing workload. Reverse-mode overhead is usually around $2.1\text{--}2.6\times$, while forward-mode overhead is around $4\text{--}7\times$.
- **Time-step scaling:** runtime grows approximately linearly with the number of local-volatility time steps N , confirming that the workload cost is dominated by repeated scan updates.
- **Memory limits:** JAX reverse-mode AD reaches practical memory limits at large M, N combinations, including $M = 250k, N = 504$ and $M = 1M, N = 252$.
- **Thread scaling:** C++ and Rust scale effectively with requested CPU threads, while JAX does not respond meaningfully to the same thread-count controls because XLA manages CPU execution internally.
- **Measurement stability:** key benchmark cells have CV values between 3.23% and 6.86%, which is acceptable for interpreting the main performance differences observed in this chapter.
- **Cloud cost-performance:** the final priced cloud comparison shows that instance choice can change both runtime and estimated per-run cost. In the stored comparison, `t2d-standard-4` is faster and lower cost per run than `n2-standard-4` for the listed best no-AD rows.
- **Budget-aware profiling:** SHA selects 24 of 48 full-grid benchmark cells, saving 50.0% of full runs while preserving 100.0% Pareto recovery and 0.0% runtime and cost regret in the final measured cloud grid.

Overall, the results show that MC+AD performance depends strongly on the interaction between workload structure, runtime system, differentiation mode, parallel execution model, and cloud cost assumptions. The local-volatility suite demonstrates that the framework can expose detailed runtime, AD, memory, and scaling trade-offs in a reproducible way, while the final cloud-profiler experiment demonstrates that cheap probes and SHA can reduce full cloud benchmarking work without losing decision quality on the measured grid.

Chapter 7

Conclusion

7.1 Summary of Work

This project designed, implemented, and evaluated a reproducible benchmarking framework for Monte Carlo option-pricing engines with automatic differentiation. The framework supports explicit workload configurations, engine abstraction, deterministic seeding, warm-up and repeated timing, analytical validation where available, SQLite result storage, environment metadata capture, cloud cost metadata, and generated report artefacts.

The completed implementation compares NumPy, JAX/XLA, C++/OpenMP, and Rust/Rayon engines across European GBM, parametric local-volatility, and Asian option workloads where supported. The local experimental campaign provides detailed evidence on validation, runtime, throughput, AD overhead, path-count scaling, time-step scaling, thread scaling, memory pressure, surface-shape sensitivity, and timing stability. The final cloud experiment extends this with cost-performance analysis, Pareto-frontier analysis, and profiler-versus-grid evaluation.

7.2 Key Findings

The results show that MC+AD performance cannot be reduced to a single backend ranking. The simple European GBM workload is useful for correctness checks, but the more demanding local-volatility and Asian workloads change the observed trade-offs. Rust/Rayon is the strongest no-AD engine in the main local-volatility results and in the highlighted European/local-volatility cloud rows, while JAX/XLA remains essential for AD and is competitive for some path-dependent configurations.

The AD results show a clear runtime-memory trade-off. Reverse-mode AD is substantially cheaper than forward mode for the scalar-output local-volatility workload at moderate sizes, but it reaches practical memory limits at larger M, N combinations. In the final cloud comparison at $M = 100,000$, JAX AD overhead ranges from $1.81\times$ for Asian reverse mode to $4.42\times$ for local-volatility forward mode.

The cloud-profiler result is the strongest final systems contribution. In the `sha_cloud_profiler_v4` experiment on `n2-standard-4`, the full grid requires 48 full benchmark cells. SHA selects 24, saving 50.0% of full runs while preserving 100.0% Pareto recovery and 0.0% runtime and cost regret relative to the full grid. The old profiler also preserves the best decisions, but selects 30 full runs, so SHA achieves the same measured decision quality with fewer full-scale evaluations.

7.3 Answering Research Questions

Hardware and software stacks. The local and cloud results show that runtime depends strongly on workload structure, engine implementation, and runtime system. Compiled CPU engines are very strong for no-AD workloads, while JAX/XLA is necessary for the reported AD measurements and can be competitive when the workload has enough structure to benefit from compilation.

Programming languages and AD libraries. C++/OpenMP and Rust/Rayon provide high no-AD throughput and explicit CPU parallelism. JAX provides the main AD capability and exposes the practical difference between forward and reverse mode. The trade-off is therefore not only language performance, but also derivative support, memory behaviour, and ease of integration into a reproducible benchmark runner.

Cloud cost-performance. The final priced comparison shows that runtime and cost should be considered together. For the measured no-AD rows at $M = 100,000$, `t2d-standard-4` is faster and lower cost per run than `n2-standard-4`. This is a measured result under the stored pricing and environment assumptions, not a universal claim about all cloud hardware.

AI/ML techniques. JAX/XLA demonstrates that compiler and AD techniques from the AI/ML ecosystem are relevant to MC workloads, especially when the computation is expressed in a form such as `jax.lax.scan` that the compiler can optimise. The project does not complete mixed precision, GPU auto-tuning, or broader hardware-aware scheduling experiments, so the conclusion is deliberately limited to the measured XLA and AD evidence.

Parallelism and bottlenecks. The thread-scaling experiments show effective scaling for C++ and Rust under OpenMP/Rayon-style parallelism, while JAX’s CPU runtime is less directly controlled by requested thread counts. The stress tests show that memory, not only arithmetic throughput, becomes a limiting factor for reverse-mode AD on large local-volatility workloads.

7.4 Future Work

The most direct next step is to broaden the cloud study. More instance types, regions, repeated cloud runs, and up-to-date pricing snapshots would test whether the SHA profiler remains reliable on larger and noisier grids. The profiler could also incorporate uncertainty estimates so that promotion decisions account explicitly for timing variance and Monte Carlo noise.

GPU execution, distributed execution, mixed precision, and hardware counters remain important extensions. The framework already has a CUDA scaffold and metadata structure, but the final evaluation does not claim GPU results. Extending the benchmark suite to GPUs and distributed workers would make the hardware-comparison part of the original proposal much stronger.

Further AD systems and languages would also be valuable. The project evaluates JAX AD, but future work could compare Enzyme, C++ AD tools, Rust AD libraries, Mojo, or domain-specific quant libraries. Finally, the frontend and API could be developed into a more complete dashboard for interactive profiler configuration and report artefact generation.

7.5 Final Remarks

Overall, the project contributes both a working engineering artefact and an empirical methodology. The artefact makes MC+AD experiments reproducible, queryable, and extensible across engines. The methodology shows how detailed local benchmarking can be combined with cloud cost metadata and successive halving to reduce expensive full-grid benchmarking while preserving measured selection quality. The final report therefore represents an expanded and completed version of the original framework work: it retains the detailed technical and experimental narrative, while adding the final cloud, cost-performance, Pareto-frontier, AD-overhead, and SHA-profiler results.

Declarations

Academic Integrity

This report is submitted as my own work. External sources, software libraries, and project guidance have been acknowledged where used. The implementation, experimental design, analysis, and final interpretation were reviewed and assembled by the author.

Generative AI Assistance

Generative AI tools were used as writing and coding assistants during parts of the project, including for drafting support, code explanation, editing suggestions, and report-merging assistance. All AI-assisted material was critically reviewed, checked against the project repository and experimental outputs, and edited by the author before inclusion. Numerical claims in the report are based on stored benchmark results and generated report artefacts rather than on unsupported model output.

Chapter 8

Appendix

8.1 Additional Results

Additional result tables and plots are generated from the stored benchmark database and CSV artefacts rather than manually transcribed. The primary generated artefacts used in the final evaluation are:

- `evaluation/tables/correctness_validation.tex`;
- `evaluation/tables/ad_overhead.tex`;
- `evaluation/tables/cloud_cost_performance.tex`;
- `evaluation/tables/profiler_comparison.tex`;
- `evaluation/figures/cloud_cost_performance_priced.png`;
- `evaluation/figures/pareto_runtime_cost_n2.png`;
- `evaluation/figures/sha_progression_n2.png`.

8.2 Configuration Files

The report artefact generator is `evaluation/generate_report_assets.py`. It reads the stored benchmark results and regenerates the tables and figures used in the final cloud evaluation. This supports the reproducibility goal of keeping report claims traceable to result files.

8.3 Extended Tables

8.4 Code Snippets

Bibliography